

ASHLEY WARD

CSCI 499

SENIOR PROJECT

4/15/2019

**IN PARTIAL FULFILLMENT OF THE
REQUIREMENTS FOR THE DEGREE OF
B.S. CYBER SECURITY**

Table of Contents

📌 Tab 1: Statement of Purpose

📌 Tab 2: Research/Background

📌 Tab 3: Hardware/Software required

📌 Tab 4: Screen Shots

📌 Tab 5: Source Code

📌 Tab 6: Test Plan

📌 Tab 7: Test Results

📌 Tab 8: Challenges Overcome

📌 Tab 9: Future Enhancements

📌 Tab 10: Presentation Slides

Statement of Purpose

Ideally, friends and family members would be able to get exactly what the receiver likes without the hassle of searching in stores for hours, possible getting a duplicate gift, or being the expensive thing, they want. People these days have had problems finding the right gift for someone, or the receiver is asking for something the giver cannot comply with because of the price or cannot find the item. On the other hand, the receivers of gifts do not know what they want or tells the same thing to multiple people causing them to get the same gift more than once. They also get stuff they do not like. A good example of this happening is when Grandmothers buy for their granddaughters. This happens because their likes and dislikes are different due to the age difference, generation gap, experiences in life and other factors.

To solve the problem, I will create the Gifting Made Easy website that allows people to go online and like different categories and input gift suggestions that will limit what they like. They will then be able to share this with their friends and family to help make it easier to find gifts that the receiver will like and are affordable because there are so many different options given with this website based on the categories. This website will save users from the problems of getting a duplicate gift or searching for hours in a store trying to

find something a friend or family member would like. It also helps the recipient to get something they did not even think about getting and from getting a duplicate gift. This website helps both sides of the party and would help all of mankind.

By completing this project, it will help me develop new skills I have not been able to enhance yet. With this project I will learn how to code from start to finish for something that could be used long term. I will also gain the knowledge and know-how of being able to make something more user friendly and something better to look at user wise. I will gain a better knowledge of the different types I am using as well as how to create a website. All in all, I will gain extensive knowledge from this project.

User Story:

Jeff has just realized he had forgotten about his best friend Jason's birthday in two days.

Jeff does not even know where to begin looking, but he remembers Jason saying something about this game that came out and looks it up. After seeing the price Jeff knew he could not afford to get that game for Jason. After surfing the web for a little bit Jeff realizes he had the Gifts Made Easy website that him and Jason shared. Jeff opens the app and has access to all of Jason's favorite things. Jeff sees Jason likes the band Pink Floyd and gets him Pink Floyd concert tickets. At Jason's birthday party, Jason opens Jeff's birthday present and

loved it. Jason was so glad he did not go with the game idea because Jason's Mom gave him that for his birthday present.

Research/Background

For my project I had to do a lot of research due to my lack of knowledge, but within my research I have learned a plethora amount of knowledge on the tools I used. To understand the project, you must understand the background and where I got the idea. When I was trying to come up with what I wanted to do, I had several different ideas such as a calendar like google or a video game. To narrow down what I wanted to do I asked some friends and family what would be useful to them. It was around Christmas time when everyone was thinking about gifts and I noticed that most of them were struggling to come up with good ideas on what to get someone else, and that is when it came to me. I need to do something that would make gifting easier for all people and generations. People would be able to share what they like and dislike or what they would like as a gift. That is where Gifting Made Easy came from.

After figuring this out I had to decide how I was going to make this possible. I knew my ultimate goal was to gather information about people that can be passed

to other people. So, I decided to do a website that will do just that. I knew from the start that I wanted to work with Python to learn more, once I decided on a website, I knew I had to use HTML combined with CSS and bootstrap to help with my user look appeal. I started down the road of creating my website, but I kept running into issues deciding how to portray it to the users. I started out with asking a lot of questions and quickly realized that it was impossible and would take me forever to come up with all these questions without using some type of AI. Therefore, I took a step back and reevaluated my plan and did some research on similar ideas, and that is where I found gift registries and came up with the category system.

The most popular ones I found were with Amazon and Zola. I played around on their websites to see what I liked and what I did not like. I really liked Amazon's style and used a lot of their ideas. They allow you to share your registry with other people so they are able to see what you chose. The difference is that with Amazon you are choosing specific items versus in my website, you are giving the buyer more freedom because your answer is broader. Amazon has a section that shows the most popular gifts and a section that has different categories that will be narrowed

down to specific items as you click through them. You are then able to add the items you like by a simple click. From there you can view your registry and share it with others. For my website I did a similar idea, but I had my categories to be a little more broader so that the buyer has more control on what they would like to buy. I also created it to where you, as the user, can see other users gift suggestions by their usernames versus you just sharing your registry.

I finally had the way I wanted to create my website; I researched many different things to come up with my different categories as well as my subcategories within my categories. Once I had all my categories I realized I needed a database to hold them all, as well as the users. I decided to go with SQL because I am most familiar with it. I started to create my modules when I came across flask, SQLAlchemy, Flask-Alchemy.

SQLAlchemy helps when inputting and connecting to a database because it takes out a lot of the SQL logic and it is an ORM or object-relational mapping tool. Flask is a framework that is actually written in Python. It is used for building a

database-driven website that deals with static pages. A static page is a page that has fixed content. Flask-Alchemy connects flask and SQLAlchemy.

These three tools together made it easier to create my modules that connect to my database and to connect to my html pages. I used Flask to help route my input to my Python that is then added to my database with SQLAlchemy. To start the process, I used flask to create an application. I used many different flask commands to connect to my html pages such as `render_template`. I also use POST and GET to gather information from my html pages. Then I created a line of code that will allow my application to run when I compile my `app.py` file. Once I compile/run the file it shows me that it is running on a port/URL that I can then click and see my website on the user side. I used SQLAlchemy to connection my application to my database. Once I got my database connected, I had to create my models also known as tables. To do this, I used SQLAlchemy in my `model.py` file. In this I created all the tables with the appropriate columns and rows. Now that I had my models setup and tables created, I was able to use `db.session.add()` and `db.session.commit()` in my Python to add or update tables in my database. Another function I learned with SQLAlchemy is

that I can query my table which will get the information from them. I can either query.all to get all of the information or I could use query.filter_by and filter in what I want. With Flask if I want to call the information from the html page, for example if I had a function

```
cookies = Types.query.all()

return render_template("home.html", cookies = cookies)
```

with this return statement, if I wanted to display the different cookies on my home page in my html I would use a for loop and it would look like this:

```
<form method="POST" action="/">    #post it to my python

    <input type="text" name="input">    #input is what the data is called

</form>

{% for cookie in cookies %}        #Loops through all the cookies

    <p> {{cookie.input}}<p>        #input is what the data is called

{% endfor %}
```

This made it really easy to go back and forth between my Python and html. Along with my HTML I used javascript as well, to pass certain information back to my Python as well as pre-form different functions such as calling different functions when a button is clicked, these functions did a range of things from highlighting the rating button to pulling up the next category of questions to be displayed.

Throughout my process of researching how to do different things like change the color on a button, or how to do something with SQLAlchemy, I came across w3schools.com which was the most beneficial and helpful site I could have ever asked for. This site taught you how to do different things in HTML, CSS, JavaScript, Python, SQLAlchemy, Flask, Flask-Alchemy. Literally everything that I was using had a tutorial and showed you how to do many other functions outside the tutorial. Once I found this site, I quickly became more knowledgeable on what I was doing and how everything connected and different ways to use the languages and tools I was using. W3schools taught me all I needed to know and more. They also have more than what I listed. I learned different things in HTML, CSS, and Bootstrap that would help my website look better on the user front. It taught me how to make all different

kinds of buttons for my categories and other buttons, as well as a data-toggle for when the user is viewing their gift suggestions or someone else's and how to do tabs for setting up my homepage to where it looks neater and more appealing. In Python, Flask, SQLAlchemy, and Flask-Alchemy, it taught me how to make the different models, how to connect the Python to my html pages, how to connect to my database and get or add information to it, as well as, different functionalities in Python itself.

Once I completed my project, I wanted to get real life users to test my website. Most of their feedback was positive and they liked the purpose of it. There was a couple of grammar errors pointed out and some things they would change such as color scheme, but overall it functioned really well, and they thought it was easy to use.

Hardware/Software

Hardware

HP Envy Computer

This is my personal computer. It has all the functionalities I needed to create and run this project.

Software

Visual Studio Code

I used Visual Studio Code because it is the source-code editor that I am most comfortable with and have used the most. It is also easy to add new packages and to run my code from Visual Studio Code. I also used it because I already had the package installed that allowed me to use Python.

Flask

After doing some research I learned about Flask, Flask is a framework that is written in Python. Flask is a framework that provides you with tools to use to create

a website. Flask provided me with many useful tools to make it easier and simpler to complete the website. For example, it made it easier for me to set up the basic templates and was used for all of my pages.

Python

I use Python as my main source of code because I really enjoy working with Python. I started learning Python when I took Data Mining with Dr. Hezekiah Babatunde. In that class I gained an interest in Python and loved how easy it was to understand. Once I completed his class, I knew I wanted to learn more about Python. This project was my chance to learn and to put my skills to the test. I have learned so much about Python throughout this project.

HTML

I used HTML because I am familiar with it from my Applied Networking Class with Dr. Yu-Ju Lin. I needed something to help me create the user side of my

website and I knew this was the best thing to use to coordinate with my JavaScript. I also knew with this, I could use CSS to help style my pages.

CSS

I used CSS to help style my website. I knew it coordinates well with HTML which was my language for creating my website.

JavaScript

I used JavaScript because I used HTML, and CSS. When you use HTML most people generally also use CSS and JavaScript. I used JavaScript to be the connection between my Python and HTML

Bootstrap

I used Bootstrap for its framework to help with the design of my website. It works well with HTML and CSS.

SQL

I use SQL because I am familiar with the database from Database Management with Dr. Valerie Sessions. I know there are other types of databases out there, but the is the one I am most familiar with and it works well with Python.

SQLAlchemy

I used SQLAlchemy with my SQL database. SQLAlchemy is a toolkit that is used with Python. SQLAlchemy allowed me to easily add and change things in my SQL database

Virtual Environment

I used a virtual environment to help keep the application dependencies away from other projects. The application dependencies are in my requirements.txt. This file must be compiled before the project will run.

Screenshots

In this session I will have many screen shots that represent my website. These screen shots will be me navigating through the website as if I am a user. It will also include some screen shots of the database to show the different tables and how the content is saved.

Figures 1.1 and 1.2 are screen shots of the homepage. It shows the different sections: How it works, Functions, Purpose, and example. The How it works sections explains to the user the steps and how to accomplish these steps. It explains to them things such as logging in and how to look at other people's suggestions. The functions section explains all the different functions that the website has. The purpose section talks about the purpose of the website and why it was created. The example section gives a story about how the website was utilized and was useful to the users.

Figure 1.1



Figure 1.2

Purpose	Example
Usually, friends and family members would be able to get exactly what the receiver likes without the hassle of searching in stores for hours, possibly getting a double gift, or being the expensive thing they want. People these days have had problems finding the right gift for someone, or the receiver is asking for something the giver can't comply with because of the price or can't find the item. On the other hand, the receivers of gifts don't know what they want or says the same thing to multiple people causing them to get the same gift more than once. They also get stuff they don't like. A good example of this happening is when Grandmothers buy for their granddaughters and other combinations. This happens because their likes and dislikes are different due to the age difference, experience in life and other factors. To solve the problem, the Gifts Were Easy website allows people to go on and answer questions that will limit down what they like. They will then be able to share this with their friends and family to help make it easier to find gifts that the receiver will like and is affordable because there are so many different options given with this website based on the questions. This website will save users from the problems of getting a double gift or searching for hours in a store trying to find something a friend or family member would like. It also helped the recipient to get something he or she even think about getting and then getting a double gift. This website helps both sides of the party and would help all of making.	Jeff has just realized he had forgotten about his best friend Jason's birthday in two days. Jeff doesn't even know where to begin looking, but he remembers Jason saying something about this game that came out and looks it up. After seeing the price Jeff knew he couldn't afford to get that game for Jason. After surfing the web for a little bit Jeff realizes he had the Gifts Were Easy website that him and Jason shared. Jeff opens the app and has access to a list of Jason's favorite things. Jeff sees Jason likes the band Pink Floyd and gets him Pink Floyd concert tickets. At Jason's birthday party, Jason opens Jeff's birthday present and loved it. Jason was so glad he didn't go with the game idea because Jason's Mom gave him that for his birthday present.

I have a Navbar that is located at the top of the page. It is located on each page. The Navbar is what allows a user to navigate to different pages. If the user is not logged in, the Navbar, figure 1.3, only has tabs, they can go to Home, Sign-up, and Login. The Home page is where the user sees figure 1.1 and 1.2. The Login page, figure 1.6, is where the user logs in. The sign-up pages, figure 1.5, it where the user is about to create an account. This Navbar is shown in figure 1.4. When a user is logged in the Navbar changes to add: Home, My Questions, Their Gift Suggestions, and Logout. The homepage is still the same as figure 1.1 and 1.2. The My Questions page, figures 1.7, 1.8, 1.9, is where the user sees the different categories. The Their Gift Suggestions, figure 1.12, is where the user can view other people's gift suggestions. The log out is a button that once clicked logs the user out, figure 1.26.

Figure 1.3

Navbar Home Sign Up Login



Figure 1.4

Navbar Home My Questions Their Gift Suggestions Logout



Figure 1.5

Home Sign Up Login

First Name
Enter First Name
We'll never share your name with anyone else.

Last Name
Enter Last Name
We'll never share your name with anyone else.

Username
Enter Username
We'll never share your username with anyone else.

Email address
Enter email
We'll never share your email with anyone else.

Password
Password
Must be eight characters long.

Register

Figure 1.6

Home Sign Up Login

Username
Enter username
We'll never share your username with anyone else.

Password
Password
Must be eight characters long.

Login

The My Questions page consists of different categories that can be clicked on to narrow down into subcategories, figure 1.11. Within all the categories and subcategories, it has a thumbs up for like and thumbs down for dislike. These are buttons associated with a number. If the user dislikes a category, it will mark its rating as 0. If the category is liked it will make its rating as 3. Once the rating button is clicked it will highlight it. It is only allowed to have one highlighted within that category. For example, food can only have a thumbs up or thumbs down this is shown in figure 1.10. When you go into a subcategory there is a previous button that will take you back to the previous category.

Figure 1.7

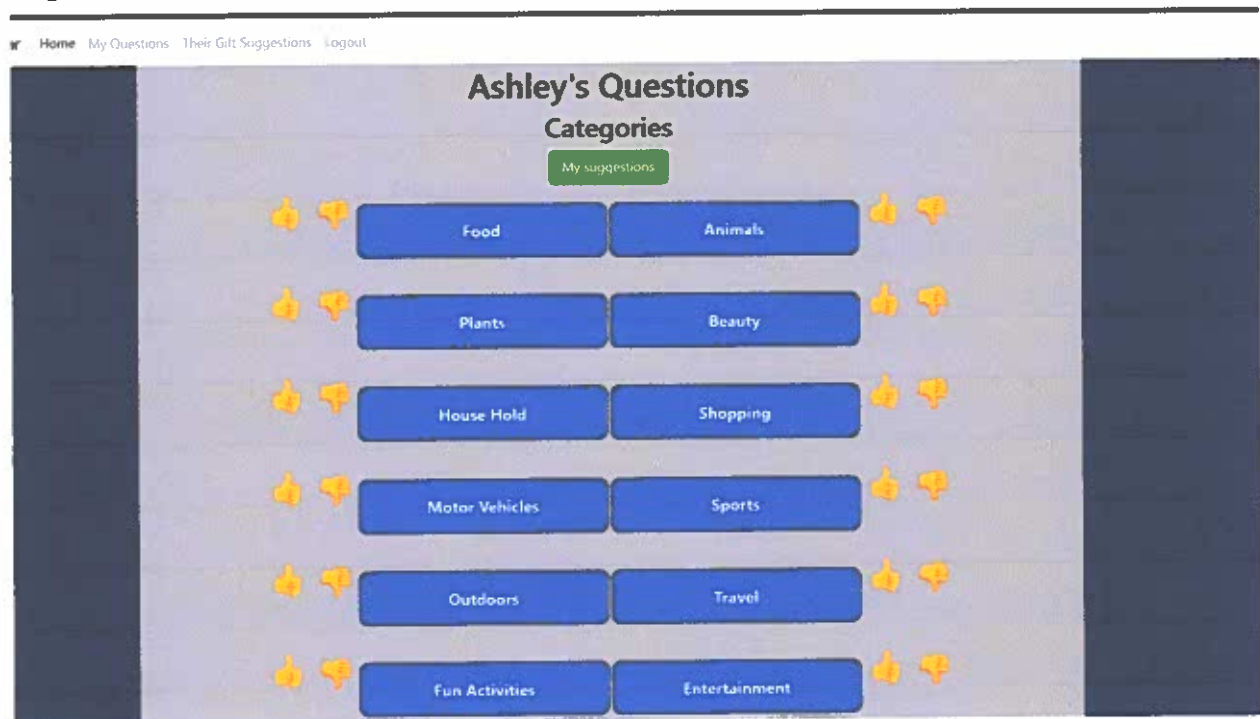


Figure 1.8

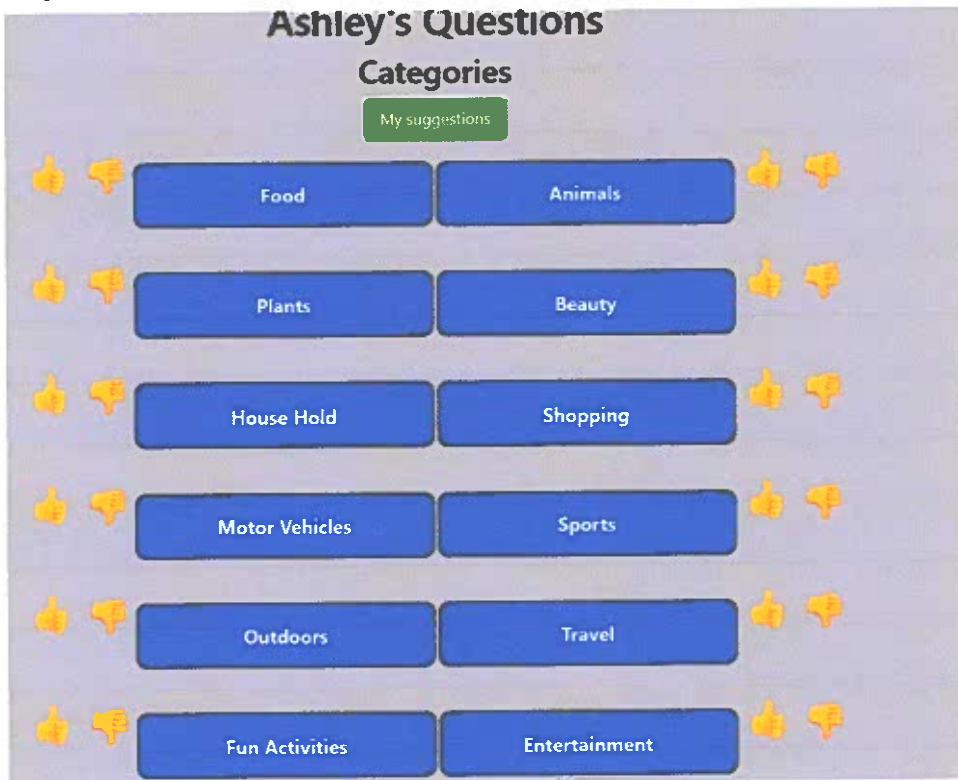


Figure 1.9

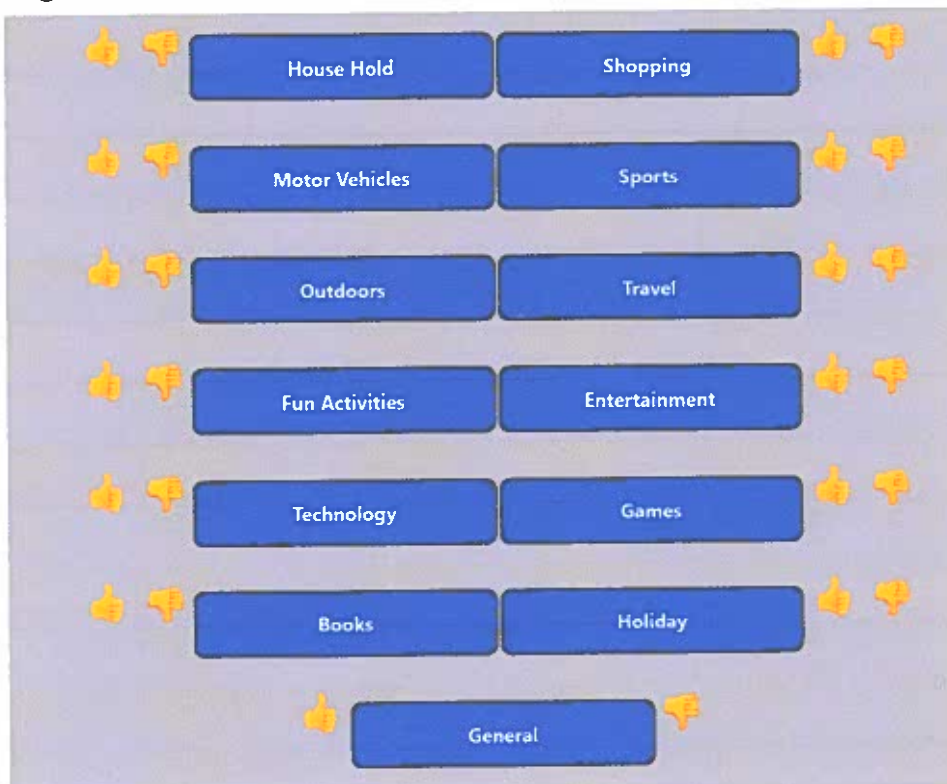


Figure 1.10

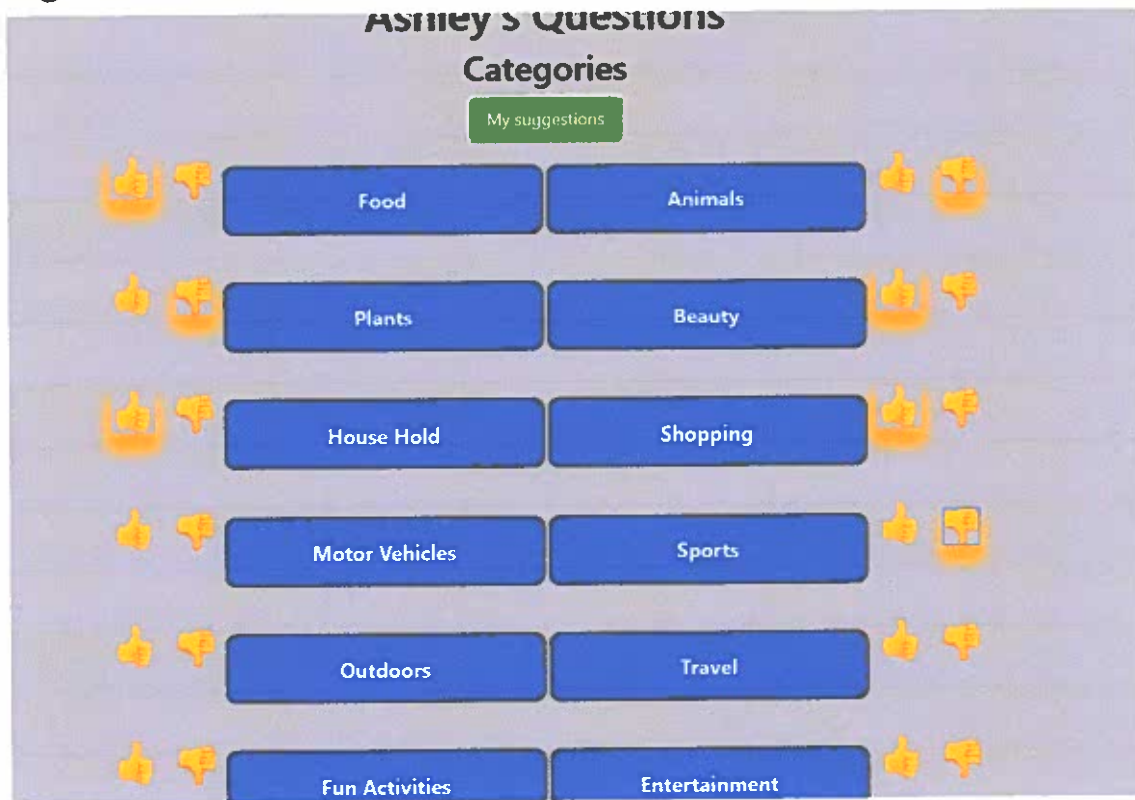
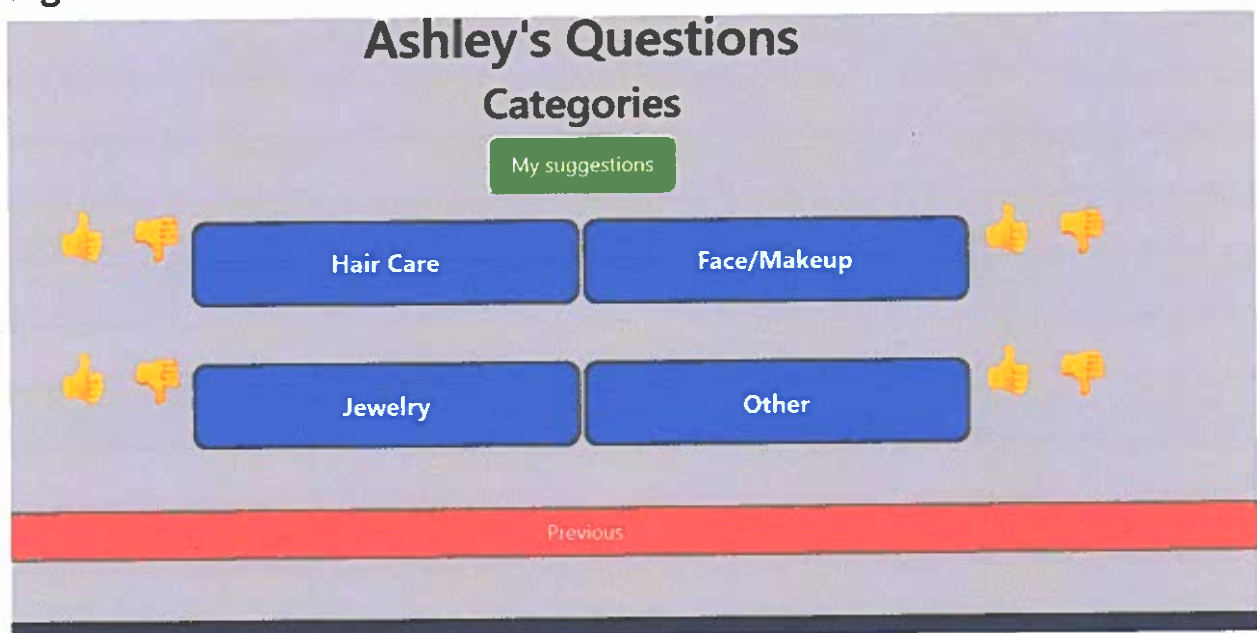


Figure 1.11



Once a user narrows down as far as the categories go, they are prompted to input a gift suggestion, figure 1.12. The user can rate the gift, once the rating is clicked it will highlight the rating, figure 1.13. Once the user clicks the save answer button, the input is then saved along with the rating. From there it is added to the given answers table, figure 1.14.

Figure 1.12

Ashley's Questions

Categories

My suggestions

Face/Makeup

Input your answer here

I have it but like it I have it but love it!!! I want it. I need it!!!

Save Answers

Previous

Given Answers

blush
face mask
hair color

Figure 1.13

Face/Makeup

Input your answer here

Face wash

☒ I have it but like it ☐ I have it but love it!!! ☐ I want it ☐ I need it!!!

Save Answers

Previous

Given Answers

- blush
- face mask
- hair color

Figure 1.14

Save Answers

Previous

Given Answers

- blush
- face mask
- hair color
- Face wash

Another function of the website is that the user can click the My Suggestions button at the top of the My questions page. This button will take the user to a page where they can view their given suggestions and the categories they liked. It is displayed with the left side the categories, the middle is where the links to search are located, the right is where the gift suggestions are displayed with the highest rating at the top and lowest at the bottom, figure 1.15. The user can click on a category and the link buttons will pull up a search. These different search buttons take you to different links such as Amazon, figure 1.18, Google, figure 1.19, Target, figure 1.20, and Walmart, figure 1.21. Once the link is clicked on it will take you to the represented website with the gift or category clicked already in the search bar to be searched.

Figure 1.15

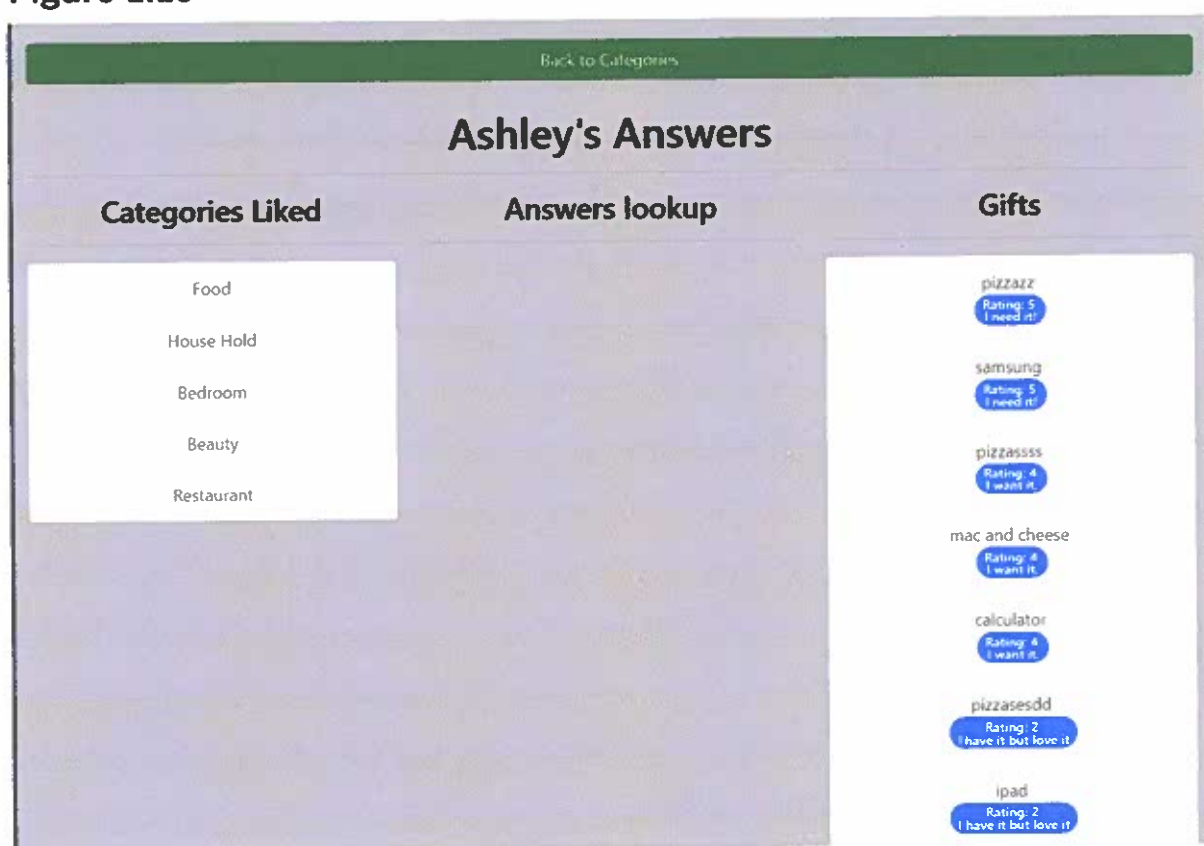


Figure 1.16

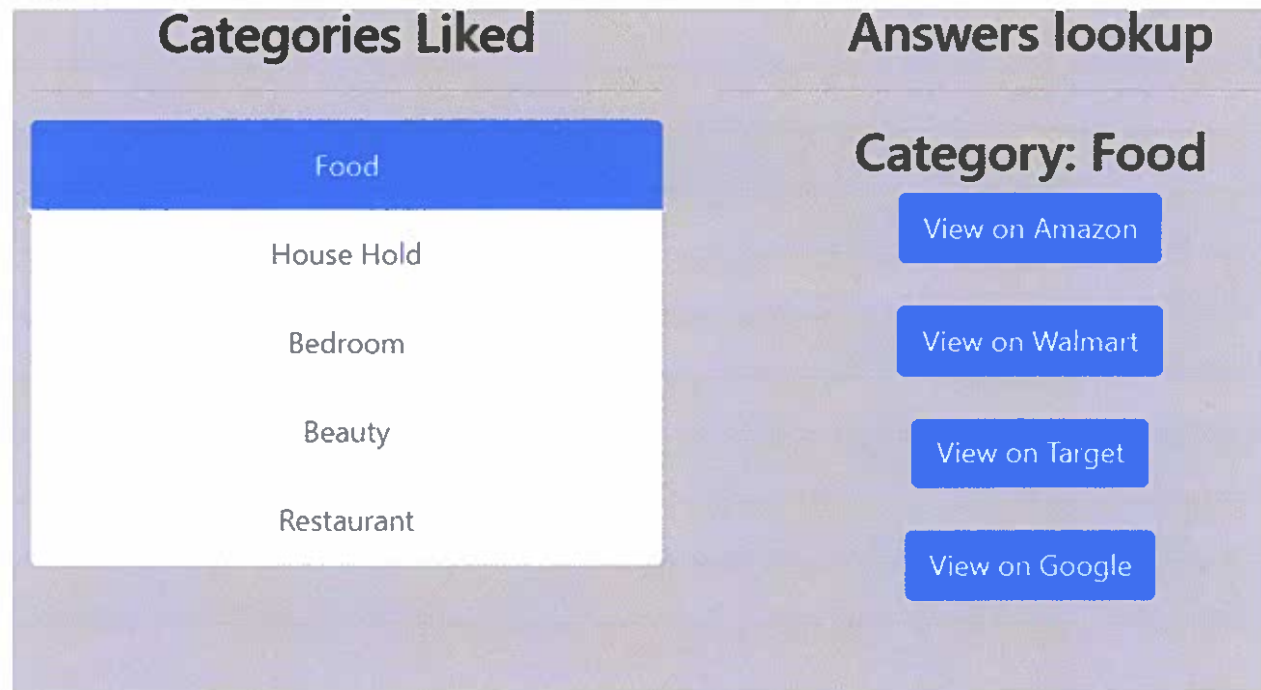


Figure 1.17

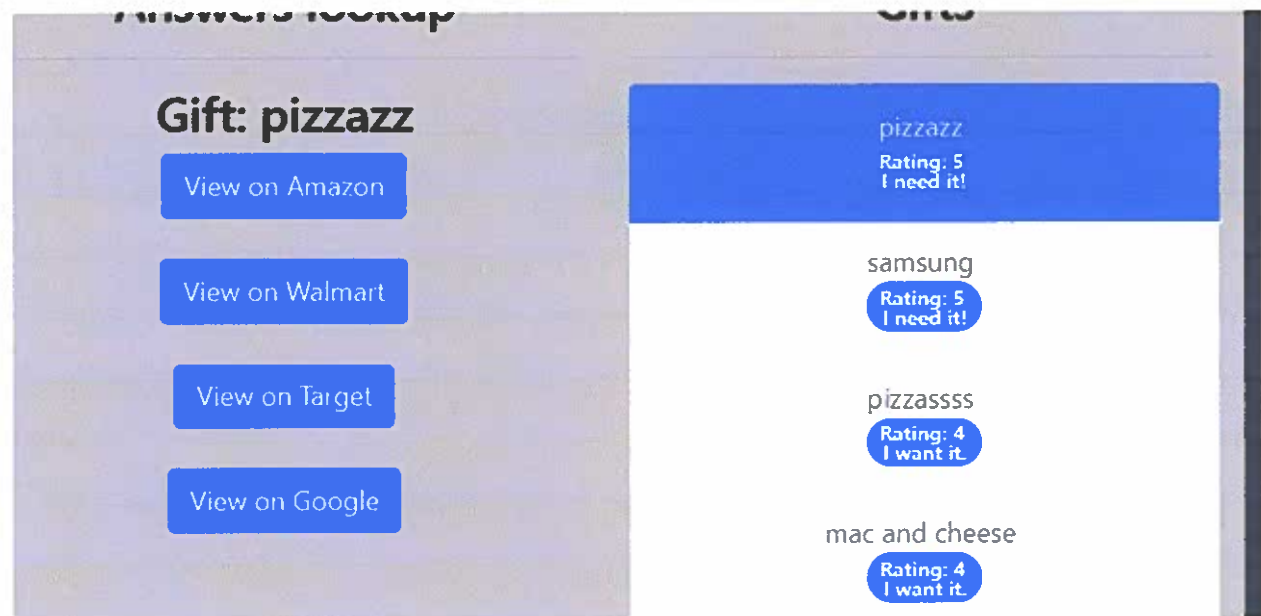


Figure 1.18

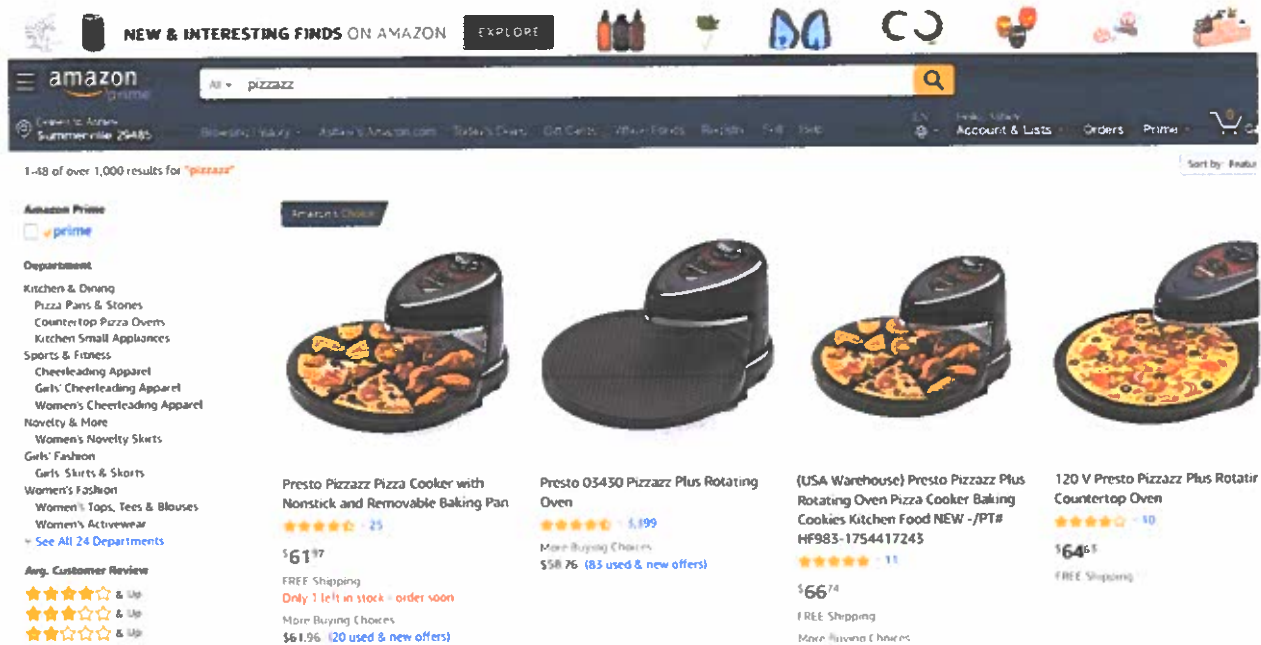


Figure 1.19

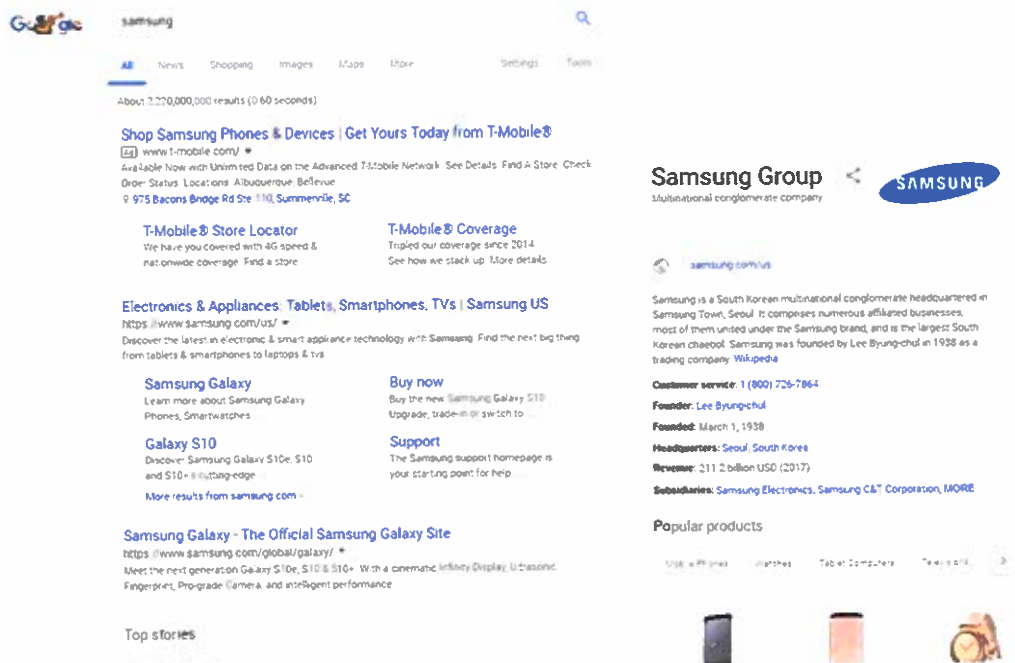


Figure 1.20

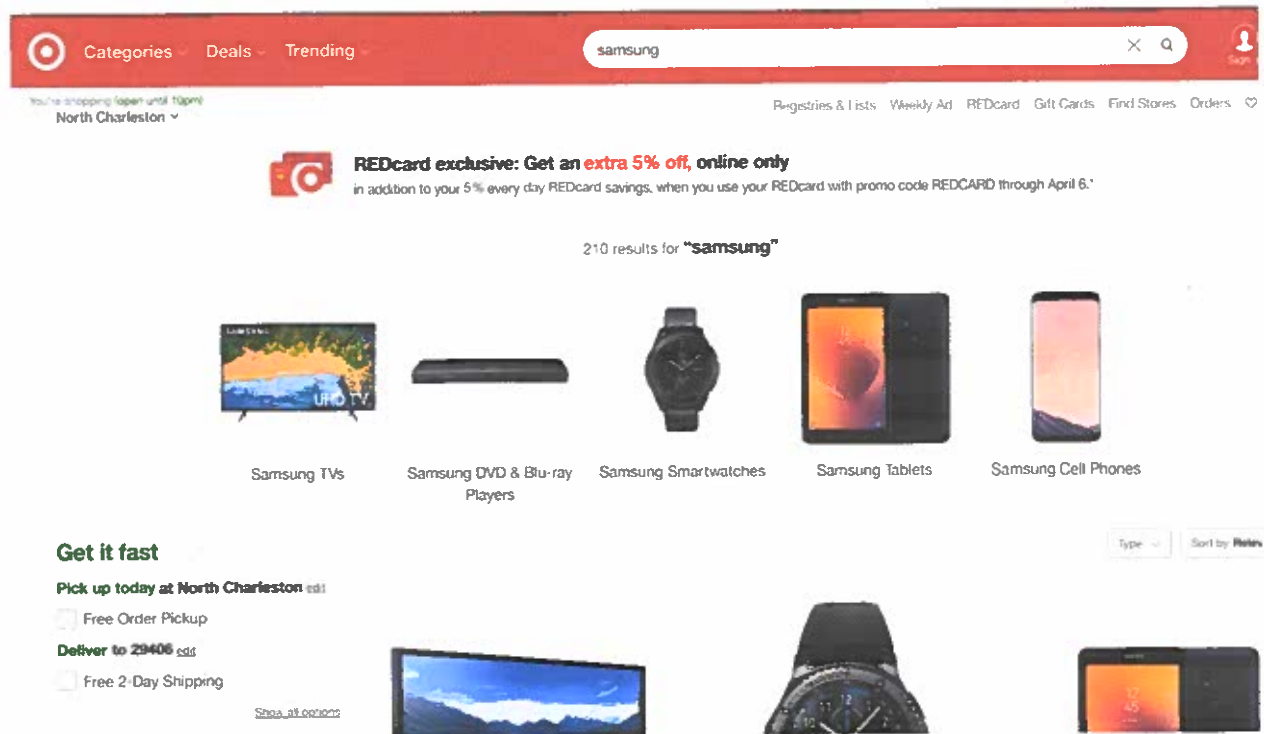
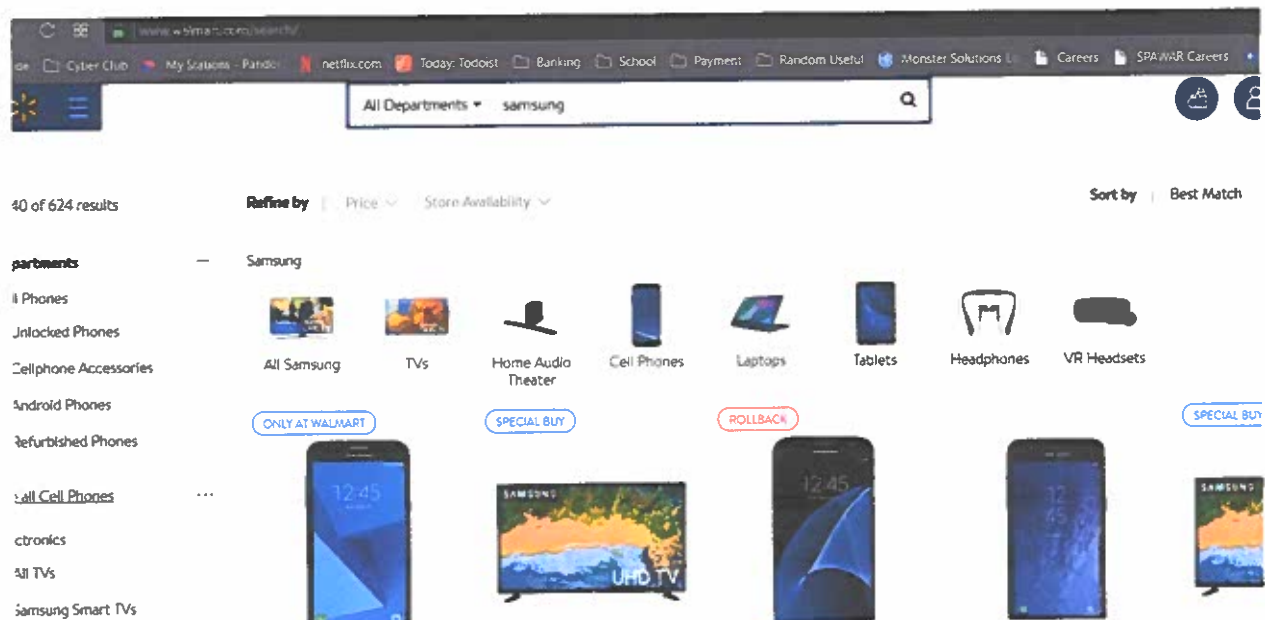


Figure 1.21



Another feature that my website has is that you are able to look up other user's gift suggestions. The user would click the Their Suggestions tab and would be prompted to put in a username, figure 1.22. Once they put in a username, they click the view button and are directed to the same page as the My suggestions button, figure 1.23, but it has the user's information that was inputted, figure 1.24. The user can then click the previous button and go back to view another user. Once they click the view button it adds the username along with the first and last name to the saved users, figure 1.25.

Figure 1.22

The screenshot shows a web interface with a light purple background. At the top, the title "View answers of someone else" is centered in bold. Below it, a subtitle "Enter the username of the person to view" is also centered. A white text input field is positioned below the subtitle. Underneath the input field is a dark green button with the word "View" in white. Below the button, the section "Saved Users" is centered in bold. Under this section, there is a list of ten entries, each showing a username followed by an arrow and a name. The entries are: "Username: ashley14ward -> Ashley Ward", "Username: branden14ward -> Ashley Ward", "Username: branden14ward -> pizza pizza", "Username: branden14ward -> Ashley Ward", "Username: branden14ward -> Ashley Ward", "Username: branden14ward -> Ashley Ward", "Username: branden14ward -> Sydney Garling", "Username: branden14ward -> Julie Henderson", "Username: pizzas -> pizza pizza", and "Username: oreo14ward -> Ashley Ward". The last entry is "Username: ward14ward -> Ashley Ward".

Saved Users	
Username: ashley14ward	-> Ashley Ward
Username: branden14ward	-> Ashley Ward
Username: branden14ward	-> pizza pizza
Username: branden14ward	-> Ashley Ward
Username: branden14ward	-> Ashley Ward
Username: branden14ward	-> Ashley Ward
Username: branden14ward	-> Sydney Garling
Username: branden14ward	-> Julie Henderson
Username: pizzas	-> pizza pizza
Username: oreo14ward	-> Ashley Ward
Username: ward14ward	-> Ashley Ward

Figure 1.23

View answers of someone else

Enter the username of the person to view

View

Saved Users

Username: ashley14ward -> Ashley Ward
Username: branden14ward -> Ashley Ward
Username: branden14ward -> pizza pizza
Username: branden14ward -> Ashley Ward
Username: branden14ward -> Ashley Ward
Username: branden14ward -> Ashley Ward
Username: branden14ward -> Sydney Garling
Username: branden14ward -> Julie Henderson
Username: pizzas -> pizza pizza
Username: oreo14ward -> Ashley Ward
Username: ward14ward -> Ashley Ward

Figure 1.24

Previous

Kaitlyn's Gift Suggestions

Categories Liked

- Food
- Animals
- Plants
- Motor Vehicles
- Shopping
- Entertainment
- Technology
- Books
- Games
- General

Gift lookup

Gift Suggestions

Cookies

Rating: 5
I need it!

Oreo

Rating: 4
I want it

Brownies

Rating: 2
I have it but love it

Figure 1.25

Saved Users	
Username:	ashley14ward -> Ashley Ward
Username:	branden14ward -> Ashley Ward
Username:	branden14ward -> pizza pizza
Username:	branden14ward -> Ashley Ward
Username:	branden14ward -> Ashley Ward
Username:	branden14ward -> Ashley Ward
Username:	branden14ward -> Sydney Garling
Username:	branden14ward -> Julie Henderson
Username:	pizzas -> pizza pizza
Username:	oreo14ward -> Ashley Ward
Username:	ward14ward -> Ashley Ward
Username:	Fake123 -> Kaitlyn Vinson

Figure 1.26



To make this website possible, I needed a database that will hold the information. To do this I set up many different tables, figure 1.27. Within these tables there are different describes. For the category table, there is ID, name, and parent_id, figure 1.28. For the Items table, there is ID, category_id, items, user_id, and rating, figure 1.29. The others table has ID, user_id, other_id, username, firstname and lastname, figure 1.30. The user table has ID, firstname, lastname, username, email, and password, figure 1.30. Throughout these tables there are multiple rows that are required and multiple that are foreign keys to other tables.

Figure 1.27

```
mysql> show tables;
+-----+
| Tables_in_seniorproject |
+-----+
| category                |
| items                   |
| others                  |
| user                    |
+-----+
4 rows in set (0.00 sec)

mysql>
```

Figure 1.28

```
mysql> describe category;
+-----+-----+-----+-----+-----+-----+
| Field      | Type          | Null | Key | Default | Extra          |
+-----+-----+-----+-----+-----+-----+
| id         | int(11)       | NO   | PRI | NULL    | auto_increment |
| name       | varchar(120)  | YES  |     | NULL    |                |
| parent_id  | int(11)       | YES  |     | NULL    |                |
+-----+-----+-----+-----+-----+-----+
3 rows in set (0.09 sec)
```


Figure 1.29

```
mysql> describe items;
```

Field	Type	Null	Key	Default	Extra
id	int(11)	NO	PRI	NULL	auto_increment
category_id	int(11)	YES	MUL	NULL	
items	varchar(120)	YES		NULL	
user_id	int(11)	NO	MUL	NULL	
rating	int(11)	YES		NULL	

```
5 rows in set (0.06 sec)
```

Figure 1.30

```
mysql> describe others;
```

Field	Type	Null	Key	Default	Extra
id	int(11)	NO	PRI	NULL	auto_increment
user_id	int(11)	NO	MUL	NULL	
other_id	int(11)	NO		NULL	
username	varchar(122)	NO		NULL	
firstname	varchar(122)	YES		NULL	
lastname	varchar(122)	YES		NULL	

```
6 rows in set (0.08 sec)
```

Figure 1.31

```
mysql> describe user;
```

Field	Type	Null	Key	Default	Extra
id	int(11)	NO	PRI	NULL	auto_increment
firstname	varchar(80)	NO		NULL	
lastname	varchar(80)	NO		NULL	
username	varchar(80)	NO	UNI	NULL	
email	varchar(120)	NO	UNI	NULL	
password	varchar(256)	NO		NULL	

```
6 rows in set (0.04 sec)
```

The category table is made up of all the categories and subcategories. Each subcategory has a parent_id on the category's id it is a sub to, Figure 1.32, 1.33, 1.34, 1.35, 1.36. The others table, figure 1.37, collects all the gift suggestions and ratings for both the gift suggestions and categories. If the item column is null that is a rating for the category, and you can find the category based off the category_id. The user_id is based off the user that added or rated the item/category. All items are connected to a category. The other table, figure 1.38, saves the username. The user_id is the id of the actual user. The other_id is the id of the user they are saving; from that id, it saves the username, first name and last name. The user table, figure 1.39, has all the created user's information. It contains the username, email, ID, password hash, first name and last name.

Figure 1.32

```
mysql> SELECT * FROM category;
```

id	name	parent_id
1	Food	0
2	Restaurant	1
3	Fast Food	2
4	Dinning in	2
5	Other	2
6	Breakfast	1
7	Lunch	1
8	Dinner	1
9	Dessert	1
10	Snack	1
11	Candy	1
12	Favorite Food	1
13	Other	1
14	Animals	0
15	Land	14
16	Air	14
17	Water	14
18	Fresh Water	17
19	Salt Water	17
20	Other	14
21	Plants	0
22	Trees	21
23	Flowers	21
24	Ferns	21
25	Other	21
26	Beauty	0
27	Hair Care	26
28	Face/Makeup	26
29	Jewelry	26
30	Other	26
31	House Hold	0
32	Bedroom	31
33	Bathroom	31
34	Living room	31
35	Dinning room	31
36	Sunroom	31
37	Laundry room	31
38	Kitchen	31
39	Office	31
40	Other	31
41	Shopping	0
42	Stores	41

Figure 1.33

43	Clothes	41
44	Shirt	43
45	Shoes	43
46	Pants	43
47	Other	43
48	Other	41
49	Motor Vehicles	0
50	Trucks	49
51	Cars	49
52	SUV	49
53	Motorcycle	49
54	RV	49
55	Dirt Bike	49
56	Four-Wheeler	49
57	Other	49
58	Sports	0
59	Favorite Team	58
60	Favorite Sport	58
61	Other	58
62	Outdoors	0
63	Water	62
64	Woods	62
65	Yard	62
66	Farming	62
67	Other	62
68	Travel	0
69	U.S. Traveling	68
70	Outside of the U.S.	68
71	Ways to vacation	68
72	Other	68
73	Fun Activities	0
74	At Home	73
75	In Town	73
76	Other	73
77	Entertainment	0
78	Movies	77
79	Action	78
80	Adventure	78
81	Animation	78
82	Biographical	78
83	Biography	78
84	Comedy	78
85	Crime	78
86	Drama	78
87	Fantasy	78
88	History	78
89	Horror	78
90	Mockumentary	78
91	Musical	78
92	Mystery	78
93	Parody	78

Figure 1.34

94	Romance	78
95	Romantic Comedy	78
96	Science Fiction	78
97	Silent	78
98	Superhero	78
99	Talk Show	78
100	Thriller	78
101	War Film	78
102	Western	78
103	Other	78
104	TV Shows	77
105	Animated Sitcom	104
106	Animates	104
107	Anthology	104
108	Comedy	104
109	Comedy-drama	104
110	Court Show	104
111	Docudrama	104
112	Documentary	104
113	Doomsday	104
114	Drama	104
115	Educational	104
116	Factual	104
117	Game Show	104
118	Legal	104
119	Medical	104
120	Mockumentary	104
121	News	104
122	Paranormal	104
123	Reality TV	104
124	Satire	104
125	Serial	104
126	Sitcom	104
127	Soap Opera	104
128	Space	104
129	Space Western	104
130	Teen Drama	104
131	Thriller	104
132	Variety	104
133	Other	104
134	Music	77
135	Alternative	134
136	Anime	134
137	Blues	134
138	Children's	134
139	Classical	134
140	Comedy	134
141	Commercial	134
142	Country	134
143	Dance	134
144	Disney	134

Figure 1.35

145	Easy Listening	134
146	Electronic	134
147	Folk Music	134
148	Fitness and Workout	134
149	Hip-Hop	134
150	Holiday	134
151	Industrial	134
152	Inspirational	134
153	Instrumental	134
154	J-Pop	134
155	Jazz	134
156	K-Pop	134
157	Karaoke	134
158	New Age	134
159	Opera	134
160	Pop	134
161	Rap	134
162	R&B	134
163	Rock	134
164	Other	134
165	Other	77
166	Technology	0
167	Phones	166
168	Computer	166
169	Tablet	166
170	Gaming System	166
171	Camera	166
172	TV	166
173	Other	166
174	Games	0
175	Board Games	174
176	Action	175
177	Adventure	175
178	Abstract Strategy	175
179	Card Games	175
180	Fantasy	175
181	Fighting	175
182	Horror	175
183	Medical	175
184	Memory	175
185	Murder	175
186	Sports	175
187	Trivia	175
188	Other	175
189	Video Games	174
190	Action	189
191	Action-Adventure	189
192	Adventure	189
193	Role Playing	189
194	Simulation	189
195	Sports	189

Figure 1.36

196	Strategy	189
197	Other	189
198	Other	174
199	Books	0
200	Art	199
201	Biographies	199
202	Business	199
203	Comics	199
204	Computer and Tech	199
205	Cooking	199
206	Education	199
207	Entertainment	199
208	Gay and Lesbian	199
209	Health and Fitness	199
210	History	199
211	Hobbies and Crafts	199
212	Home and Garden	199
213	Horror	199
214	Kids	199
215	Literature and Fiction	199
216	Medical	199
217	Music	199
218	Mysteries	199
219	Parenting	199
220	Reference	199
221	Religion	199
222	Romance	199
223	Sci-Fic and Fantasy	199
224	Science and Math	199
225	Self-Help	199
226	Social Sciences	199
227	Sports	199
228	Teen	199
229	Travel	199
230	True Crime	199
231	Westerns	199
232	Others	199
233	Holiday	0
234	Favorite Holiday	233
235	Other	233
236	General	0
237	Favorite Color	236
238	Gift Cards	236
239	Birthday	236
240	Anniversary	236
241	Other	236

241 rows in set (0.00 sec)

Figure 1.37

id	category_id	items	user_id	rating
1	1	NULL	9	0
2	21	NULL	9	0
3	31	NULL	9	0
4	49	NULL	9	3
5	62	NULL	9	3
6	73	NULL	9	3
7	166	NULL	9	3
8	199	NULL	9	3
9	14	NULL	9	0
10	26	NULL	9	0
11	41	NULL	9	3
12	58	NULL	9	0
13	68	NULL	9	3
14	77	NULL	9	3
15	174	NULL	9	3
16	233	NULL	9	3
17	236	NULL	9	3
18	4	Mexican	9	3
19	4	Italian	9	3
20	3	NULL	9	0
21	5	Gift cards to restaurants	9	4
22	234	Christmas	9	2
23	23		9	2
24	23	daisies, sunflowers, roses	9	2
25	1	NULL	1	3
26	21	NULL	1	0
27	3	pizza	1	0
28	31	NULL	1	3
29	32	NULL	1	3
30	3	pizzassss	1	4
31	14	NULL	1	0
32	26	NULL	1	3
33	3	pizza22	1	5
34	3	pizza12	1	0
35	3	pizza456	1	0
36	3	pizza489	1	0
37	3	bacon	1	0
38	3	bacon123	1	0
39	3	pizza	1	0
40	3	pizza753	1	0
41	3	pizza7895	1	0
42	3	pizza4585	1	0
43	3	pizzasessd	1	2
44	3	mac and cheese	1	4
45	3	;SELECT * FROM "user"	1	0
46	16	;SELECT	1	0
47	3	;asdf1	1	0
48	2	NULL	10	3
49	1	NULL	10	3

Figure 1.38

```
mysql> SELECT * FROM others;
```

id	user_id	other_id	username	firstname	lastname
1	5	5	ward14ward	Ashley	Ward
2	1	1	ashley14ward	Ashley	Ward
3	6	6	oreo14ward	Ashley	Ward
4	1	1	branden14ward	Ashley	Ward
5	1	4	branden14ward	pizza	pizza
6	1	5	branden14ward	Ashley	Ward
7	1	6	branden14ward	Ashley	Ward
8	1	7	branden14ward	Ashley	Ward
9	1	8	branden14ward	Sydney	Garling
10	1	9	branden14ward	Julie	Henderson
11	1	4	pizzas	pizza	pizza
12	1	6	oreo14ward	Ashley	Ward
13	10	1	ashley14ward	Ashley	Ward
14	11	5	ward14ward	Ashley	Ward
15	11	1	ashley14ward	Ashley	Ward
16	1	5	ward14ward	Ashley	Ward
17	1	11	Fake123	Kaitlyn	Vinson
18	15	1	ashley14ward	Ashley	Ward
19	16	1	ashley14ward	Ashley	Ward

19 rows in set (0.00 sec)

Figure 1.39

```
mysql> SELECT * FROM users;
```

id	first_name	last_name	username	email	password
1	Ashley	Ward	ashley14ward	ward14ward.com	phd07:sha256:50000421:1e6f053b6a:1a9898f4b:41c335711551a4a9007002551a32a27a:0b-cv2r0n1f5c:02
2	pizza	pizza	pizzas	c@kct.com	phd07:sha256:50000421:1e6f053b6a:1a9898f4b:41c335711551a4a9007002551a32a27a:0b-cv2r0n1f5c:02
3	Ashley	Ward	ashley14ward	all14ward.com	phd07:sha256:50000421:1e6f053b6a:1a9898f4b:41c335711551a4a9007002551a32a27a:0b-cv2r0n1f5c:02
4	Ashley	Ward	ashley14ward	ward14ward.com	phd07:sha256:50000421:1e6f053b6a:1a9898f4b:41c335711551a4a9007002551a32a27a:0b-cv2r0n1f5c:02
5	Ashley	Ward	branden14ward	ash14ward.com	phd07:sha256:50000421:1e6f053b6a:1a9898f4b:41c335711551a4a9007002551a32a27a:0b-cv2r0n1f5c:02
6	Sydney	Garling	branden14ward	pcys100@gmail.com	phd07:sha256:50000421:1e6f053b6a:1a9898f4b:41c335711551a4a9007002551a32a27a:0b-cv2r0n1f5c:02
7	Julie	Henderson	branden14ward	julie2@gmail.com	phd07:sha256:50000421:1e6f053b6a:1a9898f4b:41c335711551a4a9007002551a32a27a:0b-cv2r0n1f5c:02
8	Branden	Vinson	branden14ward	julie17347ab37@gmail.com	phd07:sha256:50000421:1e6f053b6a:1a9898f4b:41c335711551a4a9007002551a32a27a:0b-cv2r0n1f5c:02
9	Kaitlyn	Vinson	Fake123	test@example.com	phd07:sha256:50000421:1e6f053b6a:1a9898f4b:41c335711551a4a9007002551a32a27a:0b-cv2r0n1f5c:02
10	Ashley	Ward	ashley14ward	branden14ward.com	phd07:sha256:50000421:1e6f053b6a:1a9898f4b:41c335711551a4a9007002551a32a27a:0b-cv2r0n1f5c:02
11	oreo	oreo	oreo14ward	oreo14ward.com	phd07:sha256:50000421:1e6f053b6a:1a9898f4b:41c335711551a4a9007002551a32a27a:0b-cv2r0n1f5c:02
12	Branden	Ward	branden14ward	branden14ward.com	phd07:sha256:50000421:1e6f053b6a:1a9898f4b:41c335711551a4a9007002551a32a27a:0b-cv2r0n1f5c:02
13	Branden	Ward	branden14ward	branden14ward.com	phd07:sha256:50000421:1e6f053b6a:1a9898f4b:41c335711551a4a9007002551a32a27a:0b-cv2r0n1f5c:02
14	Branden	Ward	branden14ward	branden14ward.com	phd07:sha256:50000421:1e6f053b6a:1a9898f4b:41c335711551a4a9007002551a32a27a:0b-cv2r0n1f5c:02
15	Branden	Ward	branden14ward	branden14ward.com	phd07:sha256:50000421:1e6f053b6a:1a9898f4b:41c335711551a4a9007002551a32a27a:0b-cv2r0n1f5c:02
16	Branden	Ward	branden14ward	branden14ward.com	phd07:sha256:50000421:1e6f053b6a:1a9898f4b:41c335711551a4a9007002551a32a27a:0b-cv2r0n1f5c:02
17	Branden	Ward	branden14ward	branden14ward.com	phd07:sha256:50000421:1e6f053b6a:1a9898f4b:41c335711551a4a9007002551a32a27a:0b-cv2r0n1f5c:02
18	Branden	Ward	branden14ward	branden14ward.com	phd07:sha256:50000421:1e6f053b6a:1a9898f4b:41c335711551a4a9007002551a32a27a:0b-cv2r0n1f5c:02
19	Branden	Ward	branden14ward	branden14ward.com	phd07:sha256:50000421:1e6f053b6a:1a9898f4b:41c335711551a4a9007002551a32a27a:0b-cv2r0n1f5c:02
20	Branden	Ward	branden14ward	branden14ward.com	phd07:sha256:50000421:1e6f053b6a:1a9898f4b:41c335711551a4a9007002551a32a27a:0b-cv2r0n1f5c:02
21	Branden	Ward	branden14ward	branden14ward.com	phd07:sha256:50000421:1e6f053b6a:1a9898f4b:41c335711551a4a9007002551a32a27a:0b-cv2r0n1f5c:02

19 rows in set (0.00 sec)

```

1 from flask import Flask, render_template, redirect, request, session, jsonify
2 from models import db, User, app, Items, Category, Others
3 import re
4 import json
5 from werkzeug.security import generate_password_hash, check_password_hash
6 #imports everything needed form other pages or websites
7
8 #connects to database
9 app = Flask(__name__)
10 app.config['SQLALCHEMY_DATABASE_URI'] =
    'mysql://ashley:Branden2018Krull!@localhost/seniorproject'
11 app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False
12
13 #the user could look at the cookies and this secret_key keeps them from motifing them
    unless they know what it is.
14 app.secret_key = "this_is_a_super_secret_key"
15
16 with app.app_context():
17     db.init_app(app)
18     db.create_all()
19
20 #creates a certain way an email have to look and how long the password can be.
21 email_regex = re.compile(r"^[^@]+@[^@]+\.[^@]+$")
22 pass_regex = re.compile(r"^(.{8,})$")
23
24 #sets the initial page to the home page
25 @app.route("/")
26 def index():
27     login, user = get_user()
28     return render_template('frame.html', page='home.html', login=login, user=user)
29
30 #login page
31 @app.route("/login")
32 def loginpage():
33     login, user = get_user()
34     if login:
35         return redirect('/')
36     return render_template('frame.html', page='login.html')
37
38 #gets the username and password and compares it to the database
39 @app.route("/login/submit", methods = ["POST"])
40 def login():
41     username = request.form['username'].strip()
42     password = request.form['password'].strip()
43     #checks for ; and denys it for possible SQL injection
44     if(username.find(';') != -1):
45         return redirect('/')
46     if(password.find(';') != -1):
47         return redirect('/')
48
49     if db.session.query(User.id).filter_by(username=username).scalar() is None:
50         #Checks if user exists
51         return redirect('/login')
52     user = User.query.filter_by(username=username).first()
53     if check_password_hash(user.password, password): #Checks if the password matches
54         session_login(username)
55         return redirect('/questions')
56     return redirect('/login')
57

```

```
58 #sets user to the username that is logged in
59 def session_login(username):
60     user = User.query.filter_by(username=username).first()
61     session['user_id'] = user.id
62
63 #gets the user from the database
64 def get_user():
65     login = 'user_id' in session
66     if login:
67         return (True, User.query.filter_by(id=session['user_id']).first())
68     return (False, None)
69
70 #logs the user out and redirects to the homepage
71 @app.route('/logout')
72 def session_logout():
73     login, user = get_user()
74     if not login:
75         return redirect('/')
76     del session['user_id']
77     return redirect('/')
78
79 #register page
80 @app.route("/register")
81 def registerpage():
82     login, user = get_user()
83     if login:
84         return redirect('/')
85     return render_template('frame.html', page='register.html')
86
87 #Checks all the register variables then adds if passes all
88 @app.route("/register/submit", methods = ["POST"])
89 def register():
90     #all the information gathered by user
91     firstname = request.form['firstname'].strip()
92     lastname = request.form['lastname'].strip()
93     username = request.form['username'].strip()
94     email = request.form['email'].strip()
95     password = request.form['password'].strip()
96
97     #rejects ; for possible SQL injection
98     if(firstname.find(';') != -1):
99         return redirect('/register')
100     if(lastname.find(';') != -1):
101         return redirect('/register')
102     if(username.find(';') != -1):
103         return redirect('/register')
104     if(email.find(';') != -1):
105         return redirect('/register')
106     if(password.find(';') != -1):
107         return redirect('/register')
108
109     #checks against all the constraints
110     if not username or not password or not email or not firstname or not lastname:
111         return redirect('/register')
112     if not firstname.isalpha() or not lastname.isalpha() or not
password_regex.match(password) or not email_regex.match(email):
113         return redirect('/register')
114     if db.session.query(User.id).filter_by(email=email).scalar() is not None or
db.session.query(User.id).filter_by(username=username).scalar() is not None :
115         return redirect('/register')
```

```
116
117     #adds the user and adds a hash to their password so it is secure
118     user = User(username = username, firstname = firstname, lastname = lastname,
password = generate_password_hash(password), email = email)
119     db.session.add(user)
120     db.session.commit()
121     session_login(username)
122     return redirect('/questions')
123
124 #questions page
125 @app.route("/questions")
126 def questions():
127     login, user = get_user()
128     if not login:
129         return redirect('/')
130     return render_template('frame.html', page='questions.html', login=login,
user=user)
131
132 #gets the first set of categories
133 @app.route("/questions/1")
134 def getMyQuestions1():
135     login, user = get_user()
136     if not login:
137         return redirect('/')
138     questions = Category.query.all()
139     jsonQuestions = []
140     for name in questions:#gets the categories with a parent_id of 0
141         if name.parent_id == 0:
142             jsonQuestions.append({"name": name.name, "parent": name.parent_id, "id":
name.id})
143     return jsonify(jsonQuestions)
144
145 #get the categories depending on what the user clicks on
146 @app.route("/questions/2")
147 def getMyQuestions2():
148     id = request.args.get('ID', type=int)
149     names =request.args.get("TEXT", type=int)
150     login, user = get_user()
151     if not login:
152         return redirect('/')
153     questions = Category.query.all()
154     jsonQuestions = []
155     for name in questions:#gits the categories that have the same parent_id based on
the id that is clicked on.
156         if (name.parent_id == id):
157             jsonQuestions.append({"name": name.name, "parent": name.parent_id, "id":
name.id})
158     return jsonify(jsonQuestions)
159
160 #gets the items in the table based on the users id
161 @app.route("/questions/items")
162 def getItem():
163     login, user = get_user()
164     if not login:
165         return redirect('/')
166     notUpdated = True
167     id = request.args.get('ID', type=int)
168     text = request.args.get('TEXT', type=str)
169     rate = request.args.get('NUM', type=int)
170     items = Items.query.all()
```

```

171     it = ""
172     if(text.find(';') != -1):#rejects ; for possible SQL injection
173         return (it)
174     for item in items:
175         if(item.items == text and item.user_id == user.id):#if item already exist
updates rating
176         Items.query.filter(Items.items == text, Items.user_id ==
user.id).update({Items.rating: rate})
177         notUpdated = False
178         db.session.commit()
179     if (notUpdated):#if item doesnt exist adds the item
180         item = Items(items= text, category_id = id, user_id = user.id, rating = rate)
181         db.session.add(item)
182         it = "added"
183         db.session.commit()
184     return (it)
185
186 #Pulls in the different items associated with that cateogroy and the user's id
187 @app.route("/questions/diffItems")
188 def getDiffItem():
189     login, user = get_user()
190     if not login:
191         return redirect('/')
192     id = request.args.get('ID', type = int)
193     questions = Items.query.all()
194     jsonQuestions = []
195     for name in questions:#checks for the items that have the user's id and the
category id clicked
196         if (name.category_id == id and name.user_id == user.id):
197             jsonQuestions.append({"item": name.items, "id": name.id})
198     return jsonify(jsonQuestions)
199
200 #updates the category ratings
201 @app.route("/questions/catRating")
202 def catRating():
203     login, user = get_user()
204     if not login:
205         return redirect('/')
206     id = request.args.get('ID', type = int)
207     num = request.args.get('NUM', type = int)
208     rate = Items.query.all()
209     if not rate:#if there isn't a rating, adds the rating and category
210         item = Items(category_id = id, user_id = user.id, rating = num)
211         db.session.add(item)
212         db.session.commit()
213         return ("rating added")
214     #updates the rating to the ones that already exist
215     for name in rate:
216         if (name.user_id == user.id and name.category_id == id):
217             Items.query.filter(Items.category_id == id, Items.user_id ==
user.id).update({Items.rating: num})
218             db.session.commit()
219             return ("updated it")
220     return( "made it to cat")
221
222 #gets the previous categories that were up
223 @app.route("/questions/previous")
224 def getPrevious():
225     id = request.args.get('ID', type=int)
226     login, user = get_user()

```

```

227     if not login:
228         return redirect('/')
229     questions = Category.query.all()
230     jsonQuestions = []
231     for name in questions:#gets the categories that has the parent id that matches
the ID
232         if (name.parent_id == id):
233             jsonQuestions.append({"name": name.name, "parent": name.parent_id, "id":
name.id})
234     return jsonify(jsonQuestions)
235
236 #other page
237 @app.route("/others")
238 def otherQuestions():
239     login, user = get_user()
240     if not login:
241         return redirect('/')
242
243     return render_template('frame.html', page='others.html', login=login, user=user)
244
245 #
246 @app.route("/others/getUsername")
247 def getUsername():
248     login, user = get_user()
249     if not login:
250         return redirect('/')
251     notUpdated = True
252     userNotFound = True
253     text = request.args.get('TEXT', type=str)
254     #rejects ; for possible SQL injections
255     if(text.find(';') != -1):
256         return ("invalid username")
257     others = Others.query.all()
258     for item in others:#checks to see if the username is already saved for the user
259         if(item.user_id == user.id and item.username == text):
260             notUpdated = False
261             break
262     if(notUpdated):#adds the username for the user
263         others = User.query.all()
264         for other in others:
265             if (other.username == text):
266                 item = Others(username= text, user_id = user.id, other_id = other.id,
firstname = other.firstname, lastname = other.lastname)
267                 userNotFound = False
268                 db.session.add(item)
269                 db.session.commit()
270             if (userNotFound):#checks to see if the username exist
271                 return ("invalid username")
272     answer = Others.query.filter_by(username = text)
273     jsonQuestions = []
274     for name in answer:
275         jsonQuestions.append({"username": name.username, "firstname": name.firstname,
"lastname": name.lastname})
276     return jsonify(jsonQuestions)
277
278 #gets all the saved username associated with the user
279 @app.route("/others/getOtherUsernames")
280 def getOtherUsernames():
281     login, user = get_user()
282     if not login:

```



```

283     return redirect('/')
284     questions = Others.query.all()
285     jsonQuestions = []
286     for name in questions:#gathers the usernames associated with the user in the
table
287         if (name.user_id == user.id):
288             jsonQuestions.append({"username": name.username, "firstname":
name.firstname, "lastname": name.lastname})
289     return jsonify(jsonQuestions)
290
291 #get the answer of the username entered
292 @app.route("/others/getAnswers/<userToGet>")
293 def getAnswers(userToGet):
294     login, user = get_user()
295     if not login:
296         return redirect('/')
297     if db.session.query(User.id).filter_by(username=userToGet).scalar() is not None:
# Make sure user exists
298         userToGet = User.query.filter_by(username=userToGet)
299         jsonQuestions = []
300         categories =[]
301         for user in userToGet:
302             questions = Items.query.filter_by(user_id=user.id)
303             #passes the categories and items associated with the username entered
304             for name in questions:
305                 if (name.rating is not None and name.rating != 0):
306                     if(name.items == " "):
307                         print("blank")
308                     elif(name.items == None):
309                         cat = Category.query.filter_by(id = name.category_id)
310                         for category in cat:
311                             categories.append({"category": category.name, "rating":
name.rating, "id":name.id})
312                     else:
313                         jsonQuestions.append({"answer": name.items, "rating":
name.rating, "id":name.id})
314                         jsonQuestions = sorted(jsonQuestions, key=sortRate, reverse=True)
315                 return render_template('frame.html', page='answers.html', category =
categories, questions=jsonQuestions, login=login, user=user, userToGet=userToGet)
316 #function for the rating to be in order based on number
317 def sortRate(e):
318     return e['rating']
319
320 #gets the users answers
321 @app.route("/others/getMyAnswers/<userToGet>")
322 def getMyAnswers(userToGet):
323     login, user = get_user()
324     if not login:
325         return redirect('/')
326     if db.session.query(User.id).filter_by(username=userToGet).scalar() is not None:
# Make sure user exists
327         userToGet = User.query.filter_by(username=userToGet)
328         jsonQuestions = []
329         categories =[]
330         for user in userToGet:
331             questions = Items.query.filter_by(user_id=user.id)
332             #passes the categories and items associated with the user
333             for name in questions:
334                 if (name.rating is not None and name.rating != 0):
335                     if(name.items == " "):

```

```
336         print("blank")
337     elif(name.items == None):
338         cat = Category.query.filter_by(id = name.category_id)
339         for category in cat:
340             categories.append({"category": category.name, "rating":
name.rating, "id":name.id})
341     else:
342         jsonQuestions.append({"answer": name.items, "rating":
name.rating, "id":name.id})
343         jsonQuestions = sorted(jsonQuestions, key=sortMyRate,
reverse=True)
344     return render_template('frame.html', page='myAnswers.html', category =
categories, questions=jsonQuestions, login=login, user=user, userToGet=userToGet)
345 #function for the rating to be in order based on number
346 def sortMyRate(e):
347     return e['rating']
348
349
350     return redirect('/')
351 if __name__ == "__main__":
352     app.run(debug=True)
353
```



```
1 from flask import Flask
2 from flask_sqlalchemy import SQLAlchemy as SQL
3 from flask_migrate import Migrate
4 #connects it to my database
5 app = Flask(__name__)
6 app.config['SQLALCHEMY_DATABASE_URI'] =
    'mysql://ashley:Branden2018Krull!@localhost/mydb'
7 app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False
8
9 #connects to my app
10 db = SQL(app)
11 migrate = Migrate(app,db)
12
13
14 #Creates the user table
15 class User(db.Model):
16     id = db.Column(db.Integer, primary_key=True)
17     firstname = db.Column(db.String(80), unique=False, nullable=False)
18     lastname = db.Column(db.String(80), unique=False, nullable=False)
19     username = db.Column(db.String(80), unique=False, nullable=False)
20     email = db.Column(db.String(120), unique=True, nullable=False)
21     password = db.Column(db.String(256), unique=False, nullable=False)
22
23     def __repr__(self):
24         return '<User %r>' % self.username
25
26 #Creates the category table
27 class Category(db.Model):
28     id = db.Column(db.Integer, primary_key=True)
29     name = db.Column(db.String(120), unique=False, nullable = True)
30     parent_id = db.Column(db.Integer, db.ForeignKey('category.id'), unique=False,
    nullable = True)
31     def __repr__(self):
32         return '<Category %r - %r>' % (self.name, self.parent_id)
33
34 #Create the Others table
35 class Others(db.Model):
36     id = db.Column(db.Integer, primary_key=True)
37     user_id = db.Column(db.Integer, db.ForeignKey('user.id'), nullable=False)
38     other_id = db.Column(db.Integer, nullable = False)
39     username = db.Column(db.String(122), nullable = False)
40     firstname = db.Column(db.String(122))
41     lastname = db.Column(db.String(122))
42
43     def __repr__(self):
44         return '<Sorting %r -%r - %r>' % (self.username, self.firstname,
    self.lastname)
45
46 #Creates the items table
47 class Items(db.Model):
48     id = db.Column(db.Integer, primary_key=True)
49     category_id = db.Column(db.Integer, db.ForeignKey('category.id'), nullable=True)
50     items = db.Column(db.String(120), unique=False, nullable = True)
51     user_id = db.Column(db.Integer, db.ForeignKey('user.id'), nullable=False)
52     rating = db.Column(db.Integer, nullable = True)
53
54     def __repr__(self):
55         return '<Items %r - %r>' % (self.category_id, self.user_id)
56
57
```

```

1 <!DOCTYPE html>
2 <html>
3 <meta charset="UTF-8">
4 <meta name="viewport" content="width=device-width, initial-scale=1">
5 <link rel="stylesheet" href="https://www.w3schools.com/w3css/3/w3.css">
6 <head>
7   <header id="example1";><!--creates the pictures and background for the homepage-->
8   <style>
9     #example1 {
10       background: url(/static/img/logo01.jpg) no-repeat center;
11       background-size: cover;
12       height: 230px;
13       width: 100%;
14       position: relative;
15     }
16   </style>
17   <h1 style="text-align: center;
18     position: relative;
19     top: 52px;
20     left: 1000px;
21     transform: translate(-50%, -50%);
22     color: #4DD761;
23     font-family: comic sans ms;
24     font-size: 500%;">
25     Gifting Made Easy <img src = "/static/img/logo.jpg" style=" width: 250px;
height: 170px; float: right; margin: 22px 300px 2px 25px; " />
26   </h1 >
27
28 </header>
29 </head>
30 <body style="background-color: #111638;" >
31   <div class = "main">
32     <section class="w3-container w3-center w3-content" style="max-width: 800px">
33       <div class = "content">
34         <hr>
35         <div class="row">
36           <div class="col-6"><!--Creates all the paragraphs for the homepage-->
37             <h3 style = "font-family: comic sans ms; color: white">
38               How it Works
39             </h3>
40             <hr>
41             <div class="list-group" id="list-tab" role="tablist">
42               <p style="font-family: comic sans ms; color: white"> To start
you either need to login or sign-up found at the top in the navbar. The "My
Questions" tab will take you to different categories that
43               can be clicked on and will narrow down. The general
category is basic information such as birthday, favorite color, etc. Once you narrow
it down, you will be prompt to give a suggestion for a gift
44               related to that category. You can rate categories and all
gift suggestions that you like or dislike. If you would like to see your gift
suggestions click the green "My Suggestions" button in the
45               "My Questions" tab. The "Their Gift Suggestions" tab is
where you can look at friends and family's gift suggestion. It will prompt you to put
their username, then click view. On the left is the
46               categories they like and on the right is the gift
suggestions they have given and how they rated them. It will save the username at the
bottom; if you would like to view their suggestions again,
47               simply put their username in again.
48             </p>

```

```

49         </div>
50         <hr>
51     </div>
52     <div class="col-6">
53         <h3 style = "font-family:comic sans ms; color:white">
54             Functions
55         </h3>
56         <hr>
57         <div>
58             <p><ul style="font-family:comic sans ms; color:white; list-
59 style-type:disc; text-align: left;";>
60                 <li>Sign-up or login to begin.</li>
61                 <li>Click through the different categories to narrow them
62                 down</li>
63                 <li>Rate the different categories with a thumbs up or
64                 thumbs down</li>
65                 <li>Rate the gift suggestions you give.</li>
66                 <li>Add multiple gifts.</li>
67                 <li>Search someone else's gift suggestions.</li>
68                 <li>Save their username and look at their gift
69                 suggestions at another time.</li>
70                 <li>There is a quick shopping link to Amazon to show you
71                 the results of their gift.</li>
72             </ul></p>
73             <hr>
74         </div>
75     </div>
76     <div class="col-6">
77         <h3 style = "font-family:comic sans ms; color:white">
78             Purpose
79         </h3>
80         <hr>
81         <div >
82             <p style="font-family:comic sans ms; color:white">
83                 Ideally, friends and family members would be able to get exactly what the receiver
84                 likes without the hassle of
85                 searching in stores for hours, possible getting a
86                 double gift, or being the expensive thing they want. People
87                 these days have had problems finding the right gift
88                 for someone, or the receiver is asking for something the
89                 giver can't comply with because of the price or can't
90                 find the item. On the other hand, the receivers of gifts
91                 don't know what they want or says the same thing to
92                 multiple people causing them to get the same gift more then
93                 once. They also get stuff they don't like. A good
94                 example of this happening is when Grandmothers buy for their
95                 granddaughters and other combinations. This happens
96                 because their likes and dislikes are different due to the age
97                 difference, experience in life and other factors. To
98                 solve the problem, the Gifts Made Easy website allows people
99                 to go on and answer questions that will limit down
100                what they like. They will then be able to share this with their
101                friends and family to help make it easier to find
102                gifts that the receiver will like and is affordable because there
103                are so many different options given with this website
104                based on the questions. This website will save users from the
105                problems of getting a double gift or searching for
106                hours in a store trying to find something a friend or family member
107                would like. It also helped the recipient to get
108                something he didn't even think about getting and from getting a double

```

```
90      gift. This website helps both sides of the party and
would help all of mankind.
91      </p>
92
93      </div>
94      <hr>
95  </div>
96  <div class="col-6">
97      <h3 style = "font-family:comic sans ms; color:white">
98          Example
99      </h3>
100     <hr>
101     <div class="list-group" id="list-tab" role="tablist">
102         <p style="font-family:comic sans ms; color:white"> Jeff has
just realized he had forgotten about his best friend Jason's birthday in two days.
Jeff doesn't even know
103         where to begin looking, but he remembers Jason saying
something about this game that came out and looks it up. After
104         seeing the price Jeff knew he couldn't afford to get that
game for Jason. After surfing the web for a little bit Jeff
105         realizes he had the Gifts Made Easy website that him and
Jason shared. Jeff opens the app and has access to all of Jason's
106         favorite things. Jeff sees Jason likes the band Pink
Floyd and gets him Pink Floyd concert tickets. At Jason's birthday
107         party, Jason opens Jeff's birthday present and loved it.
Jason was so glad he didn't go with the game idea because Jason's
108         Mom gave him that for his birthday present.
109     </p>
110     </div>
111     <hr>
112 </div>
113 </div>
114 </div>
115 </div>
116 </body>
117 </html>
```

```
1 <!DOCTYPE HTML>
2 <html>
3 <head>
4   <title>Gifting Made Easy</title>
5   <!--Connects to the stylesheets needed-->
6   <link rel="stylesheet" type="text/css" href="{{ url_for('static',
filename='css/ashCustoms.css') }}">
7   <link rel="stylesheet" type="text/css" href="{{ url_for('static',
filename='css/bootstrap.min.css') }}">
8
9   <script
10     src="http://code.jquery.com/jquery-3.3.1.min.js"
11     integrity="sha256-FgpCb/KJQlLNfOu91ta32o/NMZxltwRo8QtmkMRdAu8="
12     crossorigin="anonymous">
13   </script>
14 </head>
15 <body><!--Create the Navbar at the top of the website-->
16   <nav class="navbar navbar-expand-lg navbar-light bg-light" style="z-index:999;">
17     <a class="navbar-brand" href="#">Navbar</a>
18     <button class="navbar-toggler" type="button" data-toggle="collapse" data-
target="#navbarSupportedContent"
19       aria-controls="navbarSupportedContent" aria-expanded="false" aria-
label="Toggle navigation">
20       <span class="navbar-toggler-icon"></span>
21     </button>
22
23     <div class="collapse navbar-collapse" id="navbarSupportedContent">
24       <ul class="navbar-nav mr-auto">
25         <li class="nav-item active">
26           <a class="nav-link" href="/">Home <span class="sr-only">(current)
</span></a>
27         </li>
28         {% if not login %}
29           <li><a href="/register" class="nav-link"><span ></span> Sign
Up</a></li>
30           <li><a href="/login" class="nav-link"><span ></span> Login</a></li>
31         {% else %}
32           <li class="nav-item">
33             <a class="nav-link" href="/questions">My Questions</a>
34           </li>
35           <li class="nav-item">
36             <a class="nav-link" href="/others">Their Gift Suggestions</a>
37           </li>
38           <li class="nav-item">
39             <a class="nav-link" href="/logout">Logout</a>
40           </li>
41         {% endif %}
42       </ul>
43     </div>
44   </nav>
45   <script src="{{ url_for('static', filename='js/jquery-3.3.1.min.js') }}"></script>
46   <script src="{{ url_for('static', filename='js/bootstrap.bundle.min.js') }}">
</script>
47   {% include page %}
48 </body>
49 </html>
```

```
1 <!DOCTYPE HTML>
2 <html>
3 <head>
4   <meta name="viewport" content="width=device-width, initial-scale=1.0">
5   <link href="static/css/bootstrap.min.css" rel="stylesheet" media="screen">
6 </head>
7 <body style="background-color:#111638;"><!--This is the login screen where the user
   puts in their username and password it is then sent to app.py where they are logged in
   if valid-->
8   <div class="container" id="login-frame" >
9     <form role="form" action="/login/submit" method="post" enctype="multipart/form-
       data">
10       <br>
11       <div class="form-group">
12         <label for="username">Username</label>
13         <input type="text" name="username" class="form-control" id="username" aria-
       describedby="usernameHelp" placeholder="Enter username" required>
14         <small id="usernameHelp" class="form-text text-muted">We'll never share your
       username with anyone else.</small>
15       </div>
16       <div class="form-group">
17         <label for="password">Password</label>
18         <input type="password" class="form-control" id="password" name="password"
       aria-describedby="passwordHelp" placeholder="Password" required>
19         <small id="passwordHelp" class="form-text text-muted">Must be eight
       characters long.</small>
20       </div>
21       {% if error %}
22       <p class="error"><strong>Error:</strong> {{ error }}
23       {% endif %}
24       <input type="submit" id="login-button" name="submit" class="btn btn-primary btn-
       block" value="Login"></input>
25     </form>
26     <br>
27   </div>
28 </body>
29 </html>
```

```

1 <!DOCTYPE HTML>
2 <html>
3 <head>
4
5 </head>
6 <body style="background-color:#111638;">
7     <div id="answers-frame" class="container" >
```



```

type="button" id="preview-button" class="btn btn-primary">View on Google</button></br>
</br>
44         {% endif %}
45     </div>
46     {% endfor %}
47     {% for question in questions %}<!--Allows you to click a gift
suggestion once clicked you will be redirected to amazon with a search of the selected
item-->
48         <div class="tab-pane fade" id="answer-{{question['id']}}"
role="tabpanel">
49             <h3>
50                 Gift: <span id="answer">{{ question['answer'] }}
</span>
51             </h3>
52             {% if question['answer'] != "" %}
53                 <button role="button"
onclick="window.open('https://www.amazon.com/s?k=
{{question['answer']}}&ref=nb_sb_noss_1', '_blank');" type="button" id="preview-
button" class="btn btn-primary">View on Amazon</button></br></br>
54                 <button role="button"
onclick="window.open('https://www.walmart.com/search/?cat_id=0&query=
{{question['answer']}}', '_blank');" type="button" id="preview-button" class="btn btn-
primary">View on Walmart</button></br></br>
55                 <button role="button"
onclick="window.open('https://www.target.com/s?searchTerm={{question['answer']}}',
'_blank');" type="button" id="preview-button" class="btn btn-primary">View on
Target</button></br></br>
56                 <button role="button"
onclick="window.open('https://www.google.com/#q={{question['answer']}}', '_blank');"
type="button" id="preview-button" class="btn btn-primary">View on Google</button></br>
</br>
57             {% endif %}
58         </div>
59     {% endfor %}
60 </div>
61 </div>
62 <div class="col-4">
63     <h3>
64         Gifts
65     </h3>
66     <hr>
67     <div class="list-group" id="list-tab" role="tablist">
68         {% for question in questions %} <!--Prints out the gift suggestions
based on rating and associated with the username entered and adds what they selected
for the rating based on the number-->
69         <a class="list-group-item list-group-item-action" data-
toggle="list" href="#answer-{{question['id']}}" role="tab">
70             {{ question.answer }}
71         </br>
72             {% if question.rating == 1%}
73                 <span class="badge badge-primary badge-pill">Rating:
{{ question['rating'] }} </br> I have it but like it</span>
74             {% endif %}
75             {% if question.rating == 2%}
76                 <span class="badge badge-primary badge-pill">Rating:
{{ question['rating'] }} </br> I have it but love it</span>
77             {% endif %}
78             {% if question.rating == 4%}
79                 <span class="badge badge-primary badge-pill">Rating:
{{ question['rating'] }} </br> I want it.</span>

```



```
80             {% endif %}
81             {% if question.rating == 5%}
82                 <span class="badge badge-primary badge-pill">Rating:
83     {{ question['rating'] }} </br> I need it!</span>
84             {% endif %}
85         </a>
86     {% endfor %}
87 </div>
88 </div>
89 <script>//javascript that activates the previous button to take you back
to the my questions page.
90     $("#prev-button").click(function() {
91         window.location = "/questions";
92     });
93 </script>
94 </div>
95 </div>
96 </body>
97 </html>
```

```

1 <!DOCTYPE HTML>
2 <html>
3 <head>
4
5 </head>
6 <body style="background-color:#111638;">
7     {% if login %}<!--Sets up the view for other peoples gift suggestions-->
8         <div id="other-frame" class="container" ></div>
9         <h3>
10             View answers of someone else
11         </h3>
12         <div class="form-group">
13             <div class = "form-group"><label for="user">Enter the username of
the person to view</label><input type="text" class="form-control" id="user"
placeholder=""></div>
14         </div>
15         <hr>
16         <button type="button" id="view-button" class="btn btn-success btn-
block">View</button></div>
17         <div id="table" >
18         </div>
19     </div>
20     <script src="{ { url_for('static', filename='js/home.js') } }"></script>
21     {% endif %}
22 <script>
23     var loaded = false; // Simple mutex
24     var table = $('#div#table');
25     var answers = $('#div#answers');
26     var items = "";
27     $(document).ready(function(){
28         $.get("/others", function(){
29             loadOthers();
30         });
31     });
32     //gets the other functions once page is loaded and calls the function to save the
username.
33     function loadOthers(){
34         var row = "";
35         getOtherUsernames();
36         row = "<h3>Saved Users</h3>";
37         table.append(row);
38         $("#view-button").click(function(){
39             saveUsername();
40         });
41     };
42     //saves a new username when entered if it isn't already saved
43     function saveUsername(){
44         var text = $("#user").val();
45         if(text == ""){
46
47         }
48         else{
49             $.ajax({
50                 url: "/others/getUsername",
51                 type: "get",
52                 data: {
53                     TEXT: text
54                 },
55                 success: function(response) {
56                     items = response;

```

```
57         //opens the username's gift suggestions
58         if (items != "invalid username"){
59             window.location = "/others/getAnswers/" + $("#user").val();
60         };
61     });
62 };
63 };
64 //loads the already saved users associated with the user
65 function getOtherUsernames(){
66     var row = "";
67     $.ajax({
68         url: "/others/getOtherUsernames",
69         type: "get",
70
71         success: function(response) {
72             items = response;
73             for (var i = 0; i < items.length; ++i){
74                 row = "<div class=\"container\" style=\"font-size:20px\"><tr>
75                 <td><b>Username:</b>&nbsp;"+ items[i].username + "&nbsp;<b>->
76                 </b>&nbsp;"+items[i].firstname+"&nbsp;"+items[i].lastname+"</td></tr></div>";
77                 if(items.length -1 == i){
78                     row = "<div class=\"container\" style=\"font-size:20px\">
79                     <tr><td><b>Username:</b>&nbsp;"+ items[i].username + "&nbsp;<b>->
80                     </b>&nbsp;"+items[i].firstname+"&nbsp;"+items[i].lastname+"</td></tr></div><br>";
81                 }
82                 table.append(row);
83             }
84         });
85     };
86 </script>
87 </body>
88 </html>
```

```
1 <!DOCTYPE HTML>
2 <html>
3 <head>
4
5 </head>
6 <body style="background-color:#111638;">
7     <div class="container" id="login-frame" ><!--Creates the register page with first
      name, lastname, username, email, and password it is then sent to app.py to be added to
      the database-->
8         <form role="form" action="/register/submit" method="post"
      enctype="multipart/form-data">
9             <br>
10            <div class="form-group">
11                <label for="firstname">First Name</label>
12                <input type="text" name="firstname" class="form-control"
      id="firstname" aria-describedby="firstnameHelp" placeholder="Enter First Name"
      required>
13                <small id="firstnameHelp" class="form-text text-muted">We'll never
      share your name with anyone else.</small>
14            </div>
15            <div class="form-group">
16                <label for="lastname">Last Name</label>
17                <input type="text" name="lastname" class="form-control" id="lastname"
      aria-describedby="lastnameHelp" placeholder="Enter Last Name" required>
18                <small id="lastnameHelp" class="form-text text-muted">We'll never
      share your name with anyone else.</small>
19            </div>
20            <div class="form-group">
21                <label for="username">Username</label>
22                <input type="text" name="username" class="form-control" id="username"
      aria-describedby="usernameHelp" placeholder="Enter Username" required>
23                <small id="usernameHelp" class="form-text text-muted">We'll never
      share your username with anyone else.</small>
24            </div>
25            <div class="form-group">
26                <label for="email">Email address</label>
27                <input type="email" name="email" class="form-control" id="email" aria-
      describedby="emailHelp" placeholder="Enter email" required>
28                <small id="emailHelp" class="form-text text-muted">We'll never share
      your email with anyone else.</small>
29            </div>
30            <div class="form-group">
31                <label for="password">Password</label>
32                <input type="password" class="form-control" id="password"
      name="password" aria-describedby="passwordHelp" placeholder="Password" required>
33                <small id="passwordHelp" class="form-text text-muted">Must be eight
      characters long.</small>
34            </div>
35            {% if error %}
36            <p class="error"><strong>Error:</strong> {{ error }}
37            {% endif %}
38            <input type="submit" id="register-button" name="submit" class="btn btn-
      primary btn-block" value="Register"/>
39        </form>
40        <br>
41    </div>
42 </body>
43 </html>
```

```

1 <!DOCTYPE html>
2 <html>
3 <head><!--Creates the style for the buttons-->
4
5 </head>
6 <body style="background-color:#111638;">
7
8     <div id="questions-frame" class="container">
9         <!--Puts the users name at the top-->
10        <h1>
11            {{ user.firstname }}'s Questions
12
13        </h1>
14        <div>
15            <h2>
16                Categories
17            </h2>
18            <!--Adds the button to connect to the user's suggested gifts-->
19            <button id = "button1" onclick =>My suggestions</button>
20            <script>
21                $("#button1").click(function(){
22                    window.location = "/others/getMyAnswers/" + "{{user.username}}";
23                });
24            </script>
25            <!--Where the categories will come in-->
26            <div id="table" >
27                </div>
28            </div>
29        </div>
30    </div>
31    <script>
32    var currentQuestion = 0;
33    var loaded = false; // Simple mutex
34    var questions;
35    var table = $('div#table');
36    var previousList = [0];
37    var items = "";
38    var rate = 0;
39
40
41    $(document).ready(function(){
42        $.get("/questions/1", function(data){
43            questions = data;
44            loadQuestions();
45        });
46    });
47
48    //loads in the different categories
49    function loadQuestions(){
50        loaded = false;
51        var row = ""
52        //does a for loop to make it to where it has two buttons in a row
53        for (var i = 0; i < questions.length; i++){
54            if(i == questions.length - 1 && questions[i].parent != 0){
55                row = "<div class=\"container\"> <tr> <td >" + createRatingBTN(1,
56                    questions[i].id, i) + "<button id=\"\" + questions[i].id + "\" class = \"button3\">" +
57                    questions[i].name + "</button>" + createRatingBTN(2, questions[i].id, i) + " </td>
58                    </tr></br></br> </div></br>" + createPrevBTN() + "</br></br>";
59            }
60            else if(i == questions.length - 2 && questions[i].parent != 0){

```

```

58         row = "<div class=\"container\"> <tr> <td >" + createRatingBTN(0,
questions[i].id, i) + "<button id=\"" + questions[i].id + "\" class = \"button3\">" +
questions[i].name + "</button> </td> <td ><button id=\"" + questions[i + 1].id + "\"
class = \"button3\">" + questions[i].name + "</button>" + createRatingBTN(0,
questions[i].id, i) + "</td> </tr></br></br> </div></br>" + createPrevBTN() + "</br>
</br>";
59     }
60     else if(i == questions.length -1 && questions[i].parent == 0){
61         row = "<div class=\"container\"> <tr> <td >" + createRatingBTN(1,
questions[i].id, i) + "<button id=\"" + questions[i].id + "\" class = \"button3\">" +
questions[i].name + "</button>" + createRatingBTN(2, questions[i].id, i) + "</td>
</tr></br></br> </div>";
62     }
63     else{
64         row = "<div class=\"container\"> <tr> <td >" + createRatingBTN(0,
questions[i].id, i) + "<button id=\"" + questions[i].id + "\" class = \"button3\">" +
questions[i].name + "</button> </td> <td ><button id=\"" + questions[i + 1].id + "\"
class = \"button3\">" + questions[i].name + "</button>" + createRatingBTN(0,
questions[i].id, i) + "</td> </tr></br></br> </div>";
65     }
66     table.append(row)
67 };
68 //once a category is click it gets the new ones
69 $(".button3").click(function(questions){
70     getNew(this.id, $(this).text());
71     getPrevious(false, this.id);
72 });
73 //Selects the thumbs up or deselects the thumbs up
74 $(".rate").click(function(){
75     var attrClass = "";
76     var ra = ""
77     attrClass = $(this).attr("class").split(" ");
78     for (r = 0; r < attrClass.length; r++) {
79         if (attrClass[r].includes("rating")){
80             ra = "." + attrClass[r];
81             break;
82         }
83     };
84     $(ra).removeClass('select');
85     $(this).addClass('select');
86 });
87 };
88
89 //loads in the text box page
90 function loadAnswer(id, text){
91     var row = "";
92     getOtherAnswer(id);
93     row = "</br><h3>" + text + "</h3></br><div><label for=\"answer\">Input your answer
here</label><div><input type=\"text\" class=\"form-control\" id=\"answer\"
value=\"\"> </div><button id=\"" + 1 + "\" class = \"button2 buttonRA\">I have it but
like it</button>&nbsp;&nbsp;&nbsp;<button id=\"" + 2 + "\" class = \"button2 buttonRA\">I
have it but love it!!!</button>&nbsp;&nbsp;&nbsp;<button id=\"" + 4 + "\" class =
\"button2 buttonRA\">I want it.</button>&nbsp;&nbsp;&nbsp;<button id=\"" + 5 + "\" class =
\"button2 buttonRA\">I need it!!!</button></br></br><button type=\"button\"
value=\"submit\" id=\"save-button\" onclick = \"saveItem(\" + id + "\")\" class=\"btn btn-
success btn-block\" >Save Answers</button></br>" + createPrevBTN() + "</br><h2>Given
Answers</h2>";
94     table.append(row);
95     $(".button2").click(function(){
96         updateRate(this.id)

```

```
97     $(this).removeClass('select');
98     $(this).addClass('select');
99 });
100 //button selection for rating buttons
101 $(".buttonRA").click(function(){
102     var attrClass = ""
103     var ra = ""
104     attrClass = $(this).attr("class").split(" ");
105     for (r = 0; r < attrClass.length; r++){
106         if (attrClass[r].includes("buttonRA")){
107             ra = "." + attrClass[r];
108             break;
109         }
110     };
111     $(ra).removeClass('select');
112     $(this).addClass('select');
113 });
114 };
115 //update rate
116 function updateRate(num){
117     if (num == null){
118         return rate;
119     }
120     else{
121         rate = num;
122         return rate;
123     }
124 }
125 //get the new categories from the database
126 function getNew(id, text){
127     $.ajax({
128         url: "/questions/2",
129         type: "get",
130         data: {
131             ID: id,
132             TEXT: text
133         },
134         success: function(response) {
135             table.empty();
136             if (response == 0){
137                 loadAnswer(id, text);
138             }
139             else{
140                 questions = response;
141                 loadQuestions();
142             }
143         }
144     });
145 //updates the category rating
146 function getRating(id, num){
147     $.ajax({
148         url: "/questions/catRating",
149         type: "get",
150         data: {
151             ID: id,
152             NUM: num
153         }
154     });
155 //makes the previous button functionable
156 function getPrevious(doAJAX, id){
```

```

157     if(doAJAX === true){
158         previousList.pop();
159         id = previousList[previousList.length -1];
160         $.ajax({
161             url: "/questions/2",
162             type: "get",
163             data: {
164                 ID: id
165             },
166             success: function(response) {
167                 table.empty();
168                 questions = response;
169                 loadQuestions();
170             });
171         }
172     } else{
173         previousList.push(id);
174     }
175 };
176 //create teh previous button html
177 function createPrevBTN(){
178     return "<button onclick = \"getPrevious(true)\" type=\"button\"
id=\"previous\" style=\"background-color:#D62A52\" class=\"btn btn-success btn-
block\">Previous</button>";
179 };
180 //creates the rating button html
181 function createRatingBTN(button, id, i){
182     if (button === 1){
183         return "<input onclick = \"getRating(\"+ id +\", 3)\" type =\"image\" src =
\"../static/img/up.png\" width =\"42\" height = \"42\" class= \"rating"+ i +" rate\"
>";
184     }
185     else if(button === 2){
186         return "<input onclick = \"getRating(\"+ id +\", 0)\" type =\"image\" src =
\"../static/img/down.png\" width =\"35\" height = \"35\" class= \"rating"+ i +"
rate\">";
187     }
188     else{
189         return "<input onclick = \"getRating(\"+ id +\", 3)\" type =\"image\" src =
\"../static/img/up.png\" width =\"42\" height = \"42\" class= \"rating"+ i +" rate\">
<input onclick = \"getRating(\"+ id +\", 0)\" type =\"image\" src =
\"../static/img/down.png\" width =\"35\" height = \"35\" class= \"rating"+i+"
rate\">";
190     }
191 };
192 //saves the answer to the database
193 function saveItem(id){
194     var text = $("#answer").val();
195     items = ""
196     console.log(id)
197     console.log(text)
198     console.log(rate)
199     if(text === null){
200
201     }
202     else{
203         $.ajax({
204             url: "/questions/items",
205             type: "get", //send it through get method
206             data: {

```



```
207         ID: id,
208         TEXT: text,
209         NUM: rate
210     },
211     success: function(response) {
212         items = response;
213         if (items == "added"){
214             var row = "<div class=\"container\" style=\"font-size:20px\"><tr>
215 <td>"+ text + "</td></tr></div>";
216             table.append(row);
217         }
218     });
219 }
220 };
221 //gets the other answers related to that user and the category selected
222 function getOtherAnswer(id){
223     var row = "";
224     $.ajax({
225         url: "/questions/diffItems",
226         type: "get",
227         data: {
228             ID: id
229         },
230         success: function(response) {
231             items = response;
232             for (var i = 0; i < items.length; ++i){
233                 row = "<div class=\"container\" style=\"font-size:20px\"><tr>
234 <td>"+ items[i].item + "</td></tr></div>";
235                 table.append(row);
236             }
237         });
238     };
239 </script>
240 </body>
241 </html>
```

```

1 <!DOCTYPE HTML>
2 <html>
3 <head>
4
5 </head>
6 <body style="background-color:#111638;">
7     <div id="answers-frame" class="container" >
8         </br><button type="button" id="prev-button" class="btn btn-success btn-
block">Previous</button></br>
9         <h1> <!--Adds the selected username to the top-->
10             {{ user.firstname }}'s Gift Suggestions
11         </h1>
12         <hr>
13         <div class="row">
14             <div class="col-4">
15                 <h3>
16                     Categories Liked
17                 </h3>
18                 <hr>
19                 <div class="list-group" id="list-tab" role="tablist">
20                     {% for question in category %} <!--Prints out the category based
on rating and associated with the username entered -->
21                         <a class="list-group-item list-group-item-action" data-
toggle="list" href="#category-{{question['id']}}" role="tab">
22                             {{ question['category'] }}
23                         </a>
24                     {% endfor %}
25                 </div>
26             </div>
27             <div class="col-4">
28                 <h3>
29                     Gift lookup
30                 </h3>
31                 <hr>
32                 <div class="tab-content" id="nav-tabContent">
33                     {% for question in category %} <!--Allows you to click a category
once clicked you will be redirected to amazon with a search of the selected item-->
34                         <div class="tab-pane fade" id="category-{{question['id']}}"
role="tabpanel">
35                             <h3>
36                                 Category: <span id="answer">{{ question['category'] }}
37                             </span>
38                             </h3>
39                             {% if question['category'] != "" %}
40                                 <button role="button"
onclick="window.open('https://www.amazon.com/s?k=
{{question['category']}}&ref=nb_sb_noss_1', '_blank');" type="button" id="preview-
button" class="btn btn-primary">View on Amazon</button></br></br>
41                                 <button role="button"
onclick="window.open('https://www.walmart.com/search/?cat_id=0&query=
{{question['category']}}', '_blank');" type="button" id="preview-button" class="btn
btn-primary">View on Walmart</button></br></br>
42                                 <button role="button"
onclick="window.open('https://www.target.com/s?searchTerm={{question['category']}}',
'_blank');" type="button" id="preview-button" class="btn btn-primary">View on
Target</button></br></br>
43                                 <button role="button"
onclick="window.open('https://www.google.com/#q={{question['category']}}', '_blank');"

```

```

type="button" id="preview-button" class="btn btn-primary">View on Google</button></br>
</br>
44             {% endif %}
45         </div>
46     {% endfor %}
47     {% for question in questions %} <!--Allows you to click a gift
suggestion once clicked you will be redirected to amazon with a search of the selected
item-->
48         <div class="tab-pane fade" id="answer-{{question['id']}}"
role="tabpanel">
49             <h3>
50                 Gift: <span id="answer">{{ question['answer'] }}
</span>
51             </h3>
52             {% if question['answer'] != "" %}
53                 <button role="button"
onclick="window.open('https://www.amazon.com/s?k=
{{question['answer']}}&ref=nb_sb_noss_1', '_blank');" type="button" id="preview-
button" class="btn btn-primary">View on Amazon</button></br></br>
54                 <button role="button"
onclick="window.open('https://www.walmart.com/search/?cat_id=0&query=
{{question['answer']}}', '_blank');" type="button" id="preview-button" class="btn btn-
primary">View on Walmart</button></br></br>
55                 <button role="button"
onclick="window.open('https://www.target.com/s?searchTerm={{question['answer']}}',
'_blank');" type="button" id="preview-button" class="btn btn-primary">View on
Target</button></br></br>
56                 <button role="button"
onclick="window.open('https://www.google.com/#q={{question['answer']}}', '_blank');"
type="button" id="preview-button" class="btn btn-primary">View on Google</button></br>
</br>
57             {% endif %}
58         </div>
59     {% endfor %}
60 </div>
61 </div>
62 <div class="col-4">
63     <h3>
64         Gift Suggestions
65     </h3>
66     <hr>
67     <div class="list-group" id="list-tab" role="tablist">
68         {% for question in questions %} <!--Prints out the gift suggestions
based on rating and associated with the username entered and adds what they selected
for the rating based on the number-->
69         <a class="list-group-item list-group-item-action" data-
toggle="list" href="#answer-{{question['id']}}" role="tab">
70             {{ question.answer }}
71         </br>
72             {% if question.rating == 1%}
73                 <span class="badge badge-primary badge-pill">Rating:
{{ question['rating'] }} </br> I have it but like it</span>
74             {% endif %}
75             {% if question.rating == 2%}
76                 <span class="badge badge-primary badge-pill">Rating:
{{ question['rating'] }} </br> I have it but love it</span>
77             {% endif %}
78             {% if question.rating == 4%}
79                 <span class="badge badge-primary badge-pill">Rating:
{{ question['rating'] }} </br> I want it.</span>

```

```
80             {% endif %}
81             {% if question.rating == 5%}
82                 <span class="badge badge-primary badge-pill">Rating:
83             {{ question['rating'] }} </br> I need it!</span>
84             {% endif %}
85             </a>
86         {% endfor %}
87     </div>
88     <script> //javascript that activates the previous button to take you back
89     a page so you are able to put in another user.
90     $("#prev-button").click(function() {
91         window.location = "/others";
92     });
93     </script>
94 </div>
95 </body>
96 </html>
```

```
1 html, body {
2     height: 100%;
3 }
4
5 #login-frame {
6     width: 75%;
7     position: relative;
8     left: 12.5%;
9     background-color: #AAA;
10 }
11
12 #questions-frame, #answers-frame, #home-frame, #other-frame {
13     width: 75%;
14     position: relative;
15     left: 12.5%;
16     background-color: #AAA;
17     text-align: center;
18     margin-top: -56px !important;
19     padding-top: 56px;
20 }
21
22 #answers-frame {
23     height: 100%;
24 }
25
26 .list-group {
27     overflow: auto;
28     max-height: 600px;
29 }
30 .rate { margin: 10px; }
31 .select {
32     box-shadow: 0px 12px 22px 1px orange;
33 }
34
35 #button1 {
36     cursor: pointer;
37     padding: 8px 16px;
38     border-radius: 8px;
39     text-align: center;
40     text-decoration: none;
41     display: inline-block;
42     font-size: 16px;
43     background-color: #4CAF50;
44     color: white;
45 }
46
47 #button1:hover {
48     color: #ffff00;
49     background: #000;
50     border: 1px solid #fff;
51 }
52 .button2 {
53     background-color: #0F16CD;
54     color: white;
55     padding: 10px;
56     text-align: center;
57     text-decoration: none;
58     display: inline-block;
59     font-size: 16px;
60     margin: 4px 2px;
```

```
61     cursor: pointer;
62     border-radius: 15px;
63 }
64 .button2:hover {
65     background-color: white;
66     color: black;
67     border: 2px solid #0F16CD;
68 }
69 .button3{
70     padding: 10px;
71     width: 300px;
72     height: 65px;
73     position: relative;
74     cursor: pointer;
75     font-weight: bold;
76     font-size: 120%;
77     background: #3366cc;
78     color: white;
79     border: 1px solid black;
80     border-radius: 10px;
81     display: inline-block;
82
83 }
84 .button3:hover {
85     color: #ffff00;
86     background: #000;
87     border: 1px solid #fff;
88 }
```

```
1 alembic==1.0.7
2 Click==7.0
3 Flask==1.0.2
4 Flask-Migrate==2.4.0
5 Flask-SQLAlchemy==2.3.2
6 itsdangerous==1.1.0
7 Jinja2==2.10
8 Mako==1.0.7
9 MarkupSafe==1.1.1
10 mysql==0.0.2
11 mysql-connector==2.1.6
12 mysqlclient==1.4.2.post1
13 python-dateutil==2.8.0
14 python-editor==1.0.4
15 six==1.12.0
16 SQLAlchemy==1.2.18
17 Werkzeug==0.14.1
18
```

Other Code Used

- 📄 Bootstrap.min.css
- 📄 Bootstrap.min.js
- 📄 Bootstrap.bundle.min.js
- 📄 JQuery-3.3.1.min.js

TEST PLAN For Gifts Made Easy by Ashley Ward

Test Plan Identifier:

- Master Test Plan for Gifting Made Easy

Introduction:

- The test plan will go over the different tests used to make sure each function of the website works properly
- The goals of the test plan is to show if the different functions pass or fail.

References:

- Document link to my Senior Proposal:
 - [Senior Proposal Final Update.docx](#)

Test Items:

- I will be testing the website and its functions.

Features to be Tested:

1. I will test if a user can login.
2. I will test if a user is able to register.
3. I will test if a user is able to logout.
4. I will test if a user is able to click on the different categories.
5. I will test if a user is able to use the thumbs up and thumbs down to update the rating for the categories.
6. I will test if a user is able to use the previous button to go back to the previous categories.
7. I will test if a user is able to use the tabs at the top to move from page to page.
8. I will test if a user is able to find someone else's gift suggestions once they have put their username in.
9. I will test if a user is able to use the rating buttons to update the items rating.
10. I will test if a user is able to click on other's liked categories and gift suggestions.

11. I will test if a user is able to connect to Amazon with the link provided in other's gift suggestions.
12. I will test if a user is able to connect to Google with the link provided in other's gift suggestions.
13. I will test if a user is able to connect to Walmart with the link provided in other's gift suggestions.
14. I will test if a user is able to connect to Target with the link provided in other's gift suggestions.
15. I will test if it will not let a user register if they use the same username as someone else.
16. I will test if it will not let a user log in if they do not have the right password connected to that username.
17. I will test if it will not let a user register if they do not have all the fields filled out.
18. I will test if it will not let a user register if there is not an @ or a . in email field.
19. I will test if it will not let a user register if their password is not 8 characters long.
20. I will test if a user is able to use the save button to add the gift suggestions for the categories.
21. I will test if a user can save a person's username with their first and last name next to it and go back and look at their gift suggestions later.
22. I will test if a user is able to see the logo and all the homepage info with no problem.
23. I will test if a user is able to use the "My Suggestions" button to see their gift suggestions.
24. I will check to see if it sets the rating with the highest ratings are at the top when viewing categories and gift suggestions from other users and the user's username.
25. I will test if a SQL injection can be performed.
26. I will check that the passwords are hashed in the database
27. I will check that the password being entered is being censored out.

Features Not to Be Tested:

1. None

Approach:

1. User can login.
 - a. I will use a registered username and password.
2. User is able to register.
 - a. I will fill in all the fields needed to register.
3. User is able to logout.
 - a. I will click the logout button.
4. User is able to click on the different categories.
 - a. I will click through the different categories to make sure it goes to the right one.
5. User is able to use the thumbs up and thumbs down update the rating for the categories.
 - a. I will check to make sure the rating updates in the database.
6. User is able to use the previous button to go back to the previous categories.
 - a. I will make sure the previous button goes back to the right categories.
7. User is able to use the tabs at the top to move from page to page.
 - a. I will click the different tabs to make sure it goes to the right page.
8. User is able to see someone else's gift suggestions once they have put their username in.
 - a. I will put in another username to see if the right information is pulled up.
9. User is able to use the rating buttons to update the items rating.
 - a. I will check to make sure the rating updates in the database.
10. User is able to click on other's liked categories and gift suggestions.
 - a. I will check to make sure I am able to click on the categories and gift suggestions.
11. User is able to connect to Amazon with the link provided in other's gift suggestions.
 - a. I will click the link for Amazon and make sure it connects and searches the right item.
12. User is able to connect to Google with the link provided in other's gift suggestions.

- a. I will click the link for Google and make sure it connects and searches the right item.
- 13. User is able to connect to Walmart with the link provided in other's gift suggestions.
 - a. I will click the link for Walmart and make sure it connects and searches the right item.
- 14. User is able to connect to Target with the link provided in other's gift suggestions.
 - a. I will click the link for Target and make sure it connects and searches the right item.
- 15. The website will not let a user register if they use the same username as someone else.
 - a. I will try to register with a username that is already being used.
- 16. The website will not let a user log in if they do not have the right password connected to that username.
 - a. I will try logging in with the wrong password.
- 17. The website will not let a user register if they do not have all the fields filled out.
 - a. I will try to register and leave one filled empty.
- 18. The website will not let a user register if there is not an @ or a . in email field.
 - a. I will try to register and leave out the symbols to see if it works.
- 19. The website will not let a user register if their password is not 8 characters long.
 - a. I will try to register using a password that is shorter than 8 characters long.
- 20. User is able to use the save button to add the gift suggestions for the categories.
 - a. I will attempt to hit the save button and see if it adds the item to the database and to the table below.
- 21. User can save a person's username with their first and last name next to it and go back and look at their gift suggestions later.
 - a. I will see if the username is saved and if the correct first name and last name are next to it.
- 22. User is able to see the logo and all the homepage info with no problem.
 - a. I will check the homepage to make sure everything looks correct.
- 23. User is able to use the "My Suggestions" button to see their gift suggestions.

- a. I will click the My Suggestions button to see if it goes to the right page.
- 24. Check to see if it sets the rating with the highest ratings at the top when viewing categories and gift suggestions from other users and the user's username.
 - a. I will check the database against the printed out ratings to make sure they are in order with the highest rating at the top.
- 25. I will test if a SQL injection can be performed.
 - a. I will try to perform a SQL injection to see if I am able to get information
- 26. I will check that the passwords are hashed in the database
 - a. I will create a user and check the database to see if I can read the user's password.
- 27. I will check that the password being entered is being censored out
 - a. I will type in a password.

Item Pass/Fail Criteria:

- 1. User can login.
 - a. Pass: Logs in
 - b. Fail: Does not log in
- 2. User is able to register.
 - a. Pass: It will log them in
 - b. Fail: It will not register them.
- 3. User is able to logout.
 - a. Pass: It will log out
 - b. Fail: Will not log out
- 4. User is able to click on the different categories.
 - a. Pass: Goes to the next category
 - b. Fail: Does not go to the next category
- 5. User is able to use the thumbs up and thumbs down update the rating for the categories.
 - a. Pass: Updates the rating
 - b. Fail: Does not update the rating
- 6. User is able to use the previous button to go back to the previous page.

- a. Pass: Goes to the pervious page
 - b. Fail: Does not go to the previous page
- 7. User is able to use the tabs at the top to move from page to page.
 - a. Pass: Tabs work
 - b. Fail: Tabs do not work
- 8. User is able to find someone else's gift suggestions once they have put their username in.
 - a. Pass: Pulls up the username's gift suggestions
 - b. Fail: Does not pull it up.
- 9. User is able to use the rating buttons to update the items rating.
 - a. Pass: Updates the rating.
 - b. Fail: Does not update the rating.
- 10. User is able to click on other's liked categories and gift suggestions.
 - a. Pass: Able to click on the categories or gift suggestions.
 - b. Fail: Not able to click on the categories or gift suggestions.
- 11. User is able to connect to Amazon with the link provided in other's gift suggestions.
 - a. Pass: Goes to Amazon
 - b. Fail: Does not go to Amazon
- 12. User is able to connect to Google with the link provided in other's gift suggestions.
 - a. Pass: Goes to Google
 - b. Fail: Does not go to Google
- 13. User is able to connect to Walmart with the link provided in other's gift suggestions.
 - a. Pass: Goes to Walmart
 - b. Fail: Does not go to Walmart
- 14. User is able to connect to Target with the link provided in other's gift suggestions.
 - a. Pass: Goes to Target
 - b. Fail: Does not go to Target
- 15. The Website will not let a user register if they use the same username as someone else.
 - a. Pass: Does not register them

- b. Fail: Registers them.
16. The Website will not let a user log in if they do not have the right password connected to that username.
- a. Pass: Does not allow to log in
 - b. Fail: Allows to log in.
17. The Website will not let a user register if they do not have all the fields filled out.
- a. Pass: Does not register them
 - b. Fail: Registers them.
18. The Website will not let a user register if there is not an @ or a . in email field.
- a. Pass: Does not register them
 - b. Fail: Registers them.
19. The Website will not let a user register if their password is not 8 characters long.
- a. Pass: Does not register them
 - b. Fail: Registers them
20. User is able to use the save button to add the gift suggestions for the categories.
- a. Pass: Saves the gift suggestions
 - b. Fail: Does not save the gift suggestions
21. User can save a person's username with their first and last name next to it and go back and look at their gift suggestions later.
- a. Pass: Saves and prints out the username, first name, and last name.
 - b. Fail: Does not save or printout the username, first name, and last name.
22. User is able to see the logo and all the homepage info with no problem.
- a. Pass: Can see everything.
 - b. Fail: Cannot see everything.
23. User is able to use the "My Suggestions" button to see their gift suggestions.
- a. Pass: Goes to the right page.
 - b. Fail: Does not go to the right page.
24. See if it sets the rating with the highest ratings at the top when viewing categories and gift suggestions from other users and the user's username.
- a. Pass: Are in the right order.

b. Fail: Are not in the right order.

25. I will test if a SQL injection can be performed.

a. Pass: I am unable to access information from the database.

b. Fail: I am able to access information from the database

26. I will check that the passwords are hashed in the database

a. Pass: The password is in an unreadable hash

b. Fail: I can read the password

27. I will check that the password being entered is being censored out

a. Pass: The characters are replaced with a dot.

b. Fail: You can see the password

Suspension Criteria and Resumption Requirements:

- None

Test Deliverables:

- List test deliverables, and links to them if available, including the following:
 - Test Plan (this document itself)
 - Test Cases using an actual user
 - Test Reports that tell which passed or failed

Test Environment:

- My virtual environment
- URL connected to my server

Estimate:

- None

Schedule:

- January 8, 2018 - All wanted aspects will be implemented.
- February 1, 2018 – All styling will be completed.
- March 1, 2018 – All updates and fixes will be completed.
- April 1, 2018 – All test will pass.

Staffing and Training Needs:

- None

Responsibilities:

- None

Risks:

- Could mess up SQL Database structure
- Mitigation plan has been coded to keep the structure intact when database is cleared.

Assumptions and Dependencies:

- None

Approvals:

- Dr. Sean Hayes
- Dr. Valarie Sessions

Test Results

User can login.

I logged in with the username ashley14ward and entered the incorrect password. It did not log me in and cleared the form. I logged in with the wrong username and a valid password and the form was cleared, and it did not log me in. I used a valid username combined with an invalid password from a different user and it did not log me in and cleared the form. I then entered the correct username and password, it then logged me in and directed me to the My Suggestions page.

It accomplishes this by collecting the username, it then checks the table users for that username. From there, if the username exists, it compares the password saved against the password provided. If that matches, the user is logged in as the user by saving the username and then passing it to a variable that when called, passes the logged in user. It then redirects to the My Suggestions page.

User is able to register.